



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET



Seminarski rad

**FORENZIKA FAJL SISTEMA I IMPLEMENTACIJA
ALATA ZA SKRIVANJE PODATAKA U OKVIRU FAJL
SISTEMA KORIŠĆENJEM SLACK SPACE-A**

Predmet: Digitalna forenzika

Smer: Računarstvo i informatika

Modul: Softversko inženjerstvo

Student:

Anja Tonsa Milovanović, br. ind. 1815

Mentor:

Prof. dr Bratislav Predić

Niš, jun 2026. godine

Sadržaj

1. Uvod.....	4
2. Digitalna forenzika.....	5
Digitalna forenzika se može podeliti na više specijalizovanih oblasti:	5
2.1. Osnovni principi digitalne forenzike	6
2.2. Značaj forenzičke analize	6
2.3. Proces forenzičke analize digitalnih dokaza	7
3. Osnove fajl sistema	9
3.1. Osnovne funkcije fajl sistema	9
3.2. Komponente fajl sistema.....	10
3.3. Organizacija podataka na disku	10
3.4. Alokacija prostora na disku.....	11
3.4.1. Upravljanje slobodnim prostorom i fragmentacija	11
3.4.2. Metode alokacije prostora.....	12
3.4.3. Šeme indeksiranja	14
4. Najpoznatiji fajl sistemi	16
4.1. FAT (File Allocation Table)	16
4.2. exFAT (Extended File Allocation Table).....	17
4.3. NTFS (New Technology File System).....	18
4.4. HFS (Hierarchical File System).....	19
4.5. APFS (Apple File System).....	21
4.6. Ext4 (Fourth Extended File System).....	22
5. Najčešće tehnike za skrivanje podataka u okviru fajl sistema	23
5.1. Hidden atribut fajlova	23
5.2. Skrivanje podataka u nealociranom prostoru.....	23
5.3. Alternate Data Streams	24
5.4. Steganografija na nivou fajl sistema	24
5.5. Skrivanje podataka u metapodacima fajl sistema	25
5.6. Skrivanje podataka u slack space-u	26
5.7. Poređenje tehnika skrivanja podataka.....	26
5.7.1. Poređenje efikasnosti različitih tehnika skrivanja podataka	26
5.7.2. Napredni pristupi kombinovanju tehnika skrivanja podataka	27

6. Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a	29
6.1. O alatu	29
6.2. Okruženje korišćenja alata i priprema uređaja	29
6.3. Funkcionalnosti alata	30
6.3.1. Skrivanje fajlova	30
6.3.2. Čitanje skrivenih fajlova	30
6.3.3. Brisanje skrivenih fajlova	30
6.3.4. Pregled skrivenih fajlova	30
6.4. Arhitektura sistema	30
6.4.1. DriveHandler	31
6.4.2. FileHandler	32
6.4.3. StorageManager	32
6.5. Implementacija	32
6.5.1. Pronalaženje slack prostora	32
6.5.2. Skrivanje podataka	35
6.5.3. Evidencija skrivenih podataka korišćenjem SAT tabele	38
6.5.4. Rekonstrukcija podataka	39
6.5.5. Brisanje podataka	40
6.6. CLI interfejs	40
6.7. Testiranje	41
6.8. Ograničenja	44
7. Zaključak	45
8. Reference	46

1. Uvod

U eri sveopšte digitalizacije, fajl sistemi predstavljaju jednu od ključnih komponenti svakog računarskog sistema jer omogućavaju skladištenje, organizaciju i pristup podacima na različitim uređajima, od ličnih računara do servera i velikih data centara. Kako količina digitalnih informacija neprestano raste, tako se javlja i sve veća potreba za preciznom analizom podataka radi otkrivanja potencijalnih zloupotreba i drugih bezbednosnih incidenata.

Forenzika fajl sistema predstavlja specifičnu oblast digitalne forenzike koja se bavi ispitivanjem strukture, sadržaja i aktivnosti unutar fajl sistema, sa ciljem prikupljanja relevantnih dokaza za pravne i istražne postupke. Kroz upotrebu naprednih tehnika i alata, stručnjaci mogu rekonstruisati radnje korisnika nad fajl sistemom, utvrditi vreme kreiranja ili modifikacija fajlova, kao i otkriti skrivene ili izbrisane podatke. Ova disciplina je od ključne važnosti u modernom društvu, jer doprinosi rešavanju sajber kriminala, unapređenju korporativne bezbednosti, kao i očuvanju digitalnog integriteta u različitim oblastima.

U okviru ovog rada biće istražen značaj forenzičke analize u okviru poglavlja 2, dok poglavlje 3 obrađuje osnove funkcionisanja fajl sistema. Poglavlje 4 se bavi najpoznatijim primerima fajl sistemi, a sa najčešćim tehnikama za skrivanje podataka u okviru fajl sistema upoznajemo se u poglavlju 5. Poglavlje 6 sadrži opis implementacije alata za skrivanje podataka u okviru fajl sistema korišćenjem *slack space*-a koji je razvijen za potrebe ovog rada. Poglavlje 7 predstavlja završni deo rada u kojem se iznose zaključci.

2. Digitalna forenzika

Forenzika, u najširem smislu, predstavlja primenu naučnih, tehničkih i inženjerskih principa u cilju pronalaženja, prikupljanja, očuvanja, analize i prezentovanja dokaza koji mogu imati pravnu relevantnost u sudskim i istražnim procesima. Osnovni cilj forenzike je da obezbedi objektivne, proverljive i naučno utemeljene informacije koje mogu pomoći u razjašnjavanju događaja ili aktivnosti od interesa za istragu.

Digitalna forenzika prvi put se pojavila početkom 1980-ih godina sa pojavom personalnih računara, a značajniji razvoj doživela je tokom 1990-ih. Međutim, tek početkom 21. veka zemlje poput Sjedinjenih Američkih Država formalizovale su svoje politike u oblasti digitalne forenzike. Ovaj pomak ka standardizaciji nastao je kao posledica porasta računarskog kriminala tokom 2000-ih godina i decentralizacije rada policijskih organa na nacionalnom nivou. Kako su se zločini povezani sa digitalnim uređajima povećavali, sve više ljudi je bilo uključeno u njihovo procesuiranje. Da bi se obezbedilo da se digitalni dokazi u krivičnim istragama prikupljaju i obrađuju na način koji je prihvatljiv na sudu, uvedene su posebne procedure. Danas digitalna forenzika postaje sve značajnija, a razlog za to je ogromna količina digitalnih podataka koji postoje o gotovo svakoj osobi i svemu što nas okružuje. Kako društvo sve više zavisi od računarskih sistema i *cloud* tehnologija, ljudi sve veći deo svog života provode na internetu, a ova promena obuhvata sve veći broj uređaja koji uključuju mobilne telefone, tablete, IoT uređaje i druge. Rezultat toga je ogromna količina podataka iz različitih izvora i formata. Istražitelji mogu koristiti ove digitalne dokaze za analizu i razumevanje sve šireg spektra kriminalnih aktivnosti, uključujući sajber napade, curenje podataka, kao i krivične i građanske istrage. Kao i svaki drugi dokaz, bilo fizički ili digitalni, istražitelji i organi reda moraju ga pravilno prikupiti, rukovati njime, analizirati ga i čuvati. U suprotnom, podaci mogu biti izgubljeni, izmenjeni ili postati neprihvatljivi na sudu. Stručnjaci za forenziku su odgovorni za sprovođenje digitalnih forenzičkih istraga, a kako potražnja za ovom oblasti raste, tako rastu i mogućnosti zapošljavanja. Američki Biro za statistiku rada procenjuje da će broj poslova u oblasti računarske forenzike porasti za 31% do 2029. godine [1].

Digitalna forenzika se može podeliti na više specijalizovanih oblasti:

- Forenzika računarskih sistema - analiza podataka na računarima, serverima i drugim digitalnim uređajima, uključujući fajl sisteme, memoriju i aplikacije.
- Mrežna forenzika - analiza mrežnog saobraćaja sa ciljem otkrivanja sumnjivih aktivnosti, upada ili curenja podataka.
- Forenzika mobilnih uređaja - analiza podataka sa mobilnih telefona i tableta, uključujući aplikacije, poruke, kontakte i sistemske logove.
- Forenzika oblaka - analiza podataka smeštenih u *cloud* okruženjima i virtuelnim infrastrukturnim sistemima.

- Forenzika fajl sistema - analiza strukture fajl sistema, načina alokacije prostora i oporavka, kao i istraživanje obrisanih ili skrivenih podataka, što predstavlja centralnu temu ovog rada.

Forenzika fajl sistema predstavlja naučnu disciplinu koja se bavi proučavanjem načina na koji operativni sistemi organizuju, čuvaju i upravljaju podacima na fizičkim medijumima. Poseban fokus ove oblasti je na analizi strukture fajl sistema, načinu alokacije prostora, kao i identifikaciji, oporavku i interpretaciji obrisanih ili skrivenih podataka.

Ova grana forenzike je od posebnog značaja jer omogućava razumevanje ne samo trenutnog stanja podataka na sistemu, već i njihovog prethodnog stanja, uključujući tragove aktivnosti korisnika koji nisu direktno vidljivi kroz standardne alate operativnog sistema.

2.1. Osnovni principi digitalne forenzike

Da bi forenzička analiza bila validna i prihvatljiva u pravnom kontekstu, ona mora da poštuje određene osnovne principe:

- Očuvanje integriteta podataka - Originalni podaci ne smeju biti izmenjeni tokom analize, svi postupci moraju da se sprovode na kopijama ili forenzičkim slikama ispitivanog medijuma.
- Dokumentovanje procesa - Svaki korak analize mora biti detaljno dokumentovan, uključujući korišćene alate, metode i vremenske oznake.
- Ponovljivost - Analiza mora da može da se ponovi i mora da se dođe do istih rezultata, koristeći iste podatke i metode.
- Prihvatljivost u sudskom procesu - Svi prikupljeni dokazi i rezultati analize moraju biti validni, proverljivi i u skladu sa pravnim i proceduralnim standardima.

Ovi principi obezbeđuju da digitalni dokazi zadrže svoju verodostojnost i da mogu biti korišćeni u pravnim postupcima bez sumnje u njihovu autentičnost.

2.2. Značaj forenzičke analize

Forenzička analiza fajl sistema ima značajan praktični i teorijski doprinos u savremenom digitalnom okruženju.

Sa stanovišta praktične primene, forenzička analiza fajl sistema igra ključnu ulogu u brojnim oblastima. U oblasti kriminalistike, ova analiza omogućava otkrivanje kritičnih dokaza u slučajevima sajber kriminala, finansijskih prevara, neovlašćenog pristupa sistemima i čak terorističkih aktivnosti. Istovremeno, u zaštiti intelektualne svojine, forenzička analiza je neophodna za otkrivanje industrijske špijunaže, krađe poverljivih podataka i neovlašćenog kopiranja vrednog sadržaja. Razumevanjem načina na koji se podaci mogu sakriti, manipulirati

ili obrisati, organizacije su u mogućnosti da razviju bolje zaštitne mehanizme i pojačaju sigurnost svojih informacionih sistema.

Pored toga, forenzička analiza ima praktičnu primenu u oporavku podataka. Omogućava pronalaženje i vraćanje podataka koji su slučajno obrisani, izgubljeni usled hardverske greške ili oštećenja. Ovaj aspekt je od posebnog značaja za organizacije koje se suočavaju sa gubitkom kritičnih informacija. Na nivou unutrašnjih organizacijskih procesa, forenzička analiza fajl sistema se koristi za detaljnu analizu aktivnosti zaposlenih, otkrivanje mogućih zloupotreba sistema ili neovlašćenog pristupa poverljivim podacima, što je čini važnim alatom u korporativnom upravljanju i zaštiti.

Forenzička analiza doprinosi razvoju formalnih standarda i metodologija na međunarodnom nivou. Kroz sistematičko proučavanje fajl sistema i forenzičkih procedura, definišu se jasni standardi za rukovanje elektronskim dokazima, što omogućava konzistentnost i pouzdanost u digitalnoj forenzici širom sveta. Ovi standardi su od vitalnog značaja za priznavanje digitalnih dokaza u pravnom sistemu i za obezbeđivanje da forenzičke analize budu prihvaćene kao validne i pouzdane u sudskim postupcima.

2.3. Proces forenzičke analize digitalnih dokaza

Proces forenzičke analize digitalnih dokaza predstavlja strukturiran skup aktivnosti koje se sprovode sa ciljem identifikacije, prikupljanja, očuvanja, analize i interpretacije digitalnih podataka koji mogu imati dokaznu vrednost. Ovaj proces mora biti strogo kontrolisan i standardizovan kako bi se obezbedila verodostojnost i pravna prihvatljivost rezultata.

Digitalna forenzika se ne zasniva samo na tehničkoj analizi, već i na metodološkom pristupu koji osigurava integritet podataka tokom celog procesa istrage. Nacionalni institut za standarde i tehnologiju (NIST) definiše četiri koraka u procesu digitalne forenzičke analize:

1. Prikupljanje podataka - Identifikuju se digitalni uređaji ili mediji za skladištenje koji sadrže podatke, metapodatke ili druge digitalne informacije relevantne za forenzičku istragu. U krivičnim slučajevima, organi reda zaplenjuju dokaze sa mesta potencijalnog zločina kako bi se obezbedio striktan lanac čuvanja dokaza. Kako bi se očuvao integritet dokaza, forenzički timovi prave forenzičku kopiju podataka pomoću uređaja za kloniranje hard diskova ili specijalnih forenzičkih alata. Nakon procesa kopiranja, originalni podaci se bezbedno čuvaju, dok se dalja istraga sprovodi nad kopijama kako bi se izbegla bilo kakva izmena originala.
2. Ispitivanje podataka - Istražitelji detaljno analiziraju podatke i metapodatke u potrazi za tragovima sajber kriminalnih aktivnosti. Forenzički analitičari mogu da povrate digitalne podatke iz različitih izvora, uključujući istoriju *web browser*-a, logove komunikacije, udaljene uređaje za skladištenje i obrisane ili dostupne delove diska. Takođe mogu izdvojiti informacije iz keš memorije operativnog sistema i praktično bilo kog drugog dela računarskog sistema.
3. Analiza podataka - Forenzički analitičari koriste različite metodologije i digitalne forenzičke alate kako bi izvukli podatke i zaključke iz digitalnih dokaza. Na primer, za

otkrivanje skrivenih podataka ili metapodataka mogu koristiti specijalizovane tehnike, kao što je analiza u realnom vremenu, koja ispituje aktivne sisteme radi pronalaženja volatilnih podataka. Takođe mogu primeniti reverznu steganografiju, metodu koja otkriva podatke skrivene pomoću steganografije, tehnike koja sakriva informacije unutar naizgled običnih poruka. Istražitelji mogu koristiti i sopstvene ili *open-source* alate kako bi povezali nalaze sa konkretnim napadačima ili akterima pretnji.

4. Izveštavanje - Kada se istraga završi, forenzički stručnjaci izrađuju formalni izveštaj koji prikazuje analizu, uključujući šta se dogodilo i ko bi mogao biti odgovoran. Izveštaji se razlikuju u zavisnosti od slučaja. Kod sajber kriminala, oni mogu sadržati i preporuke za otklanjanje ranjivosti kako bi se sprečili budući napadi. Takođe se često koriste za predstavljanje digitalnih dokaza na sudu i dele se sa organima reda, osiguravajućim kućama, regulatornim telima i drugim nadležnim institucijama.

Proces forenzičke analize digitalnih dokaza predstavlja rigorozan i strukturiran pristup koji osigurava da se digitalni tragovi pravilno identifikuju, sačuvaju i interpretiraju. Dosledna primena svih faza procesa omogućava pouzdane rezultate i obezbeđuje pravnu validnost digitalnih dokaza [1].

3. Osnove fajl sistema

Fajl sistem predstavlja strukturu i set pravila koja operativni sistem koristi za organizovanje, čuvanje, pronalaženje i pristupanje podacima na fizičkim medijumima kao što su hard diskovi, SSD diskovi, USB memorije i druga skladišta.

Osnovna uloga fajl sistema je da omogući apstrakciju između korisnika i fizičkog načina na koji se podaci zapisuju na medijumu. Fajl sistem obezbeđuje logički pregled kroz fajlove i direktorijume i na taj način se kompleksnost hardverskog skladištenja skriva, a korisniku se omogućava jednostavan i intuitivan pristup podacima.

3.1. Osnovne funkcije fajl sistema

Osnovne funkcije fajl sistema uključuju skup mehanizama koji omogućavaju efikasno, organizovano i bezbedno upravljanje podacima na memorijskim uređajima. Ove funkcije obezbeđuju apstrakciju između korisnika i fizičkog načina na koji se podaci skladište na disku.

Fajl sistem organizuje fajlove u direktorijume čime se stvara hijerarhijska struktura nalik stablu. Ova struktura počinje od *root* direktorijuma koji se dalje grana na poddirektorijume, čime se kreiraju logički putevi do specifičnih fajlova. Ovakav model organizacije omogućava korisnicima i aplikacijama intuitivnu navigaciju kroz sistem fajlova, gde se svaki fajl može identifikovati pomoću jedinstvene putanje. Na ovaj način se apstrahuje kompleksnost fizičkog skladištenja, jer korisnik ne mora da zna gde se podaci fizički nalaze na disku, već im pristupa preko logičke strukture.

Fajl sistem upravlja raspodelom prostora na fizičkom medijumu tako što definiše kako se slobodni prostor dodeljuje, koristi i oslobađa. Ovo se ostvaruje kroz različite algoritme alokacije koji prate stanje slobodnih i zauzetih delova diska. Fajl sistem održava interne strukture, kao što su tabele alokacije ili bitmape slobodnog prostora, koje povezuju logičke delove fajla sa fizičkim lokacijama na disku.

Fajl sistem implementira mehanizme kontrole pristupa koji definišu koji korisnik može da čita, menja ili izvršava određene fajlove i direktorijume. Ovi mehanizmi se zasnivaju na korisničkim i grupnim dozvolama, kao i na dodatnim sigurnosnim politikama koje zavise od operativnog sistema. Kontrola pristupa obezbeđuje zaštitu podataka od neovlašćenih izmena i pristupa, čime se povećava sigurnost sistema. U savremenim fajl sistemima, kao što su NTFS ili ext4, ove dozvole su detaljno definisane i omogućavaju finu granularnost u dodeli privilegija različitim korisnicima i procesima.

Fajl sistem čuva metapodatke, odnosno dodatne informacije o fajlovima koje ne predstavljaju njihov sadržaj, već opisuju njihove osobine i status. To uključuje informacije kao što su veličina fajla, vremenske oznake (vreme kreiranja, poslednje izmene i poslednjeg pristupa), vlasništvo nad fajlom, kao i dozvole pristupa. Metapodaci se skladište odvojeno od samog sadržaja fajla, u posebnim strukturama fajl sistema, što omogućava brzu obradu i pristup informacijama bez

potrebe za čitanjem celog fajla. Kako bi se obezbedila konzistentnost i integritet metapodataka, savremeni fajl sistemi koriste mehanizme kao što su *journaling*, *copy-on-write* pristup i kontrolne sume. Ovi mehanizmi omogućavaju oporavak sistema u slučaju grešaka, neočekivanog prekida rada ili oštećenja podataka.

Fajl sistemi omogućavaju efikasno i brzo pronalaženje podataka kroz upotrebu različitih indeksnih struktura i algoritama pretrage. Ove strukture omogućavaju da se fajlovi lociraju bez potrebe za sekvencijalnim pregledom celog diska. U zavisnosti od tipa fajl sistema, koriste se različiti pristupi optimizacije pretrage, kao što su B-stabla, *hash* tabele ili specijalizovane strukture za direktorijume i metapodatke. Zahvaljujući ovim mehanizmima, pristup fajlovima ostaje brz čak i na sistemima sa velikim brojem podataka.

3.2. Komponente fajl sistema

Fajl sistem se sastoji od nekoliko ključnih komponenti koje zajedno omogućavaju upravljanje skladištenjem podataka.

Fajl je osnovna jedinica podataka u organizaciji bilo kog fajl sistema. Sadrži povezane podatke kao jedinstvenu celinu, bez obzira da li je u pitanju dokument, fotografija ili izvršni program.

Direktorijum ili folder predstavlja specijalnu vrstu strukture koja se koristi kao kontejner za fajlove, čime se omogućava njihova organizacija u okviru strukture koja olakšava pristup podacima i navigaciju.

Metapodaci fajlova predstavljaju skup dodatnih informacija koje opisuju karakteristike i osobine fajla, ali ne sadrže sam njegov sadržaj. Oni omogućavaju fajl sistemu i operativnom sistemu da upravljaju fajlovima na efikasan i organizovan način, bez potrebe za direktnim pristupom njihovim podacima.

Kontrolni blok fajla predstavlja strukturu podataka koju fajl sistem koristi za čuvanje svih relevantnih metapodataka o pojedinačnom fajlu. On funkcioniše kao interna evidencija fajla unutar operativnog sistema i omogućava brzo i efikasno upravljanje fajlovima tokom njihovog životnog ciklusa. Kontrolni blok fajla sadrži informacije kao što su ime fajla, atributi, veličina, lokacija na disku, pokazivači na blokove podataka, kao i informacije o dozvolama i vremenskim oznakama.

3.3. Organizacija podataka na disku

Organizacija podataka na disku predstavlja način na koji fajl sistem fizički i logički raspoređuje informacije na memorijskom medijumu. Iako korisnik posmatra podatke kroz fajlove i direktorijume, u pozadini se svi podaci zapisuju na nizovima sektora, blokova ili klastera koji predstavljaju jedinice za skladištenje podataka.

Osnovna fizička jedinica skladištenja je sektor, koji predstavlja stvarnu fizičku jedinicu za skladištenje na hardverskom nivou. Svaki sektor najčešće ima veličinu od 512 bajtova ili 4 KB u savremenim diskovima. Više sektora se grupiše u veće logičke jedinice koje se nazivaju

blokovi ili klasteri, a koji predstavljaju osnovnu jedinicu alokacije prostora u fajl sistemu. Kod Windows operativnog sistema standardna veličina klastera je 4KB, što je ekvivalentno sa 8 sektora tradicionalne veličine.

Fajl sistem mapira logičke fajlove na fizičke lokacije na disku, pri čemu jedan fajl može zauzimati jedan ili više klastera. U zavisnosti od veličine fajla, poslednji klaster često nije u potpunosti popunjen, što dovodi do pojave neiskorišćenog prostora poznatog kao *slack space*.

3.4. Alokacija prostora na disku

Alokacija prostora na disku predstavlja proces kojim fajl sistem dodeljuje slobodan prostor fajlovima koji se kreiraju ili modifikuju. Cilj ovog procesa je optimalno korišćenje dostupnog prostora, uz obezbeđivanje brzog pristupa podacima i minimalne fragmentacije.

3.4.1. Upravljanje slobodnim prostorom i fragmentacija

Fajl sistem održava posebne strukture za praćenje slobodnog prostora na disku, čime omogućava efikasno dodeljivanje klastera prilikom kreiranja i proširivanja fajlova. U zavisnosti od implementacije, najčešće se koriste bitmape, tabele alokacije ili liste slobodnih blokova. Ove strukture omogućavaju operativnom sistemu brzo pronalaženje dostupnih klastera i optimizaciju procesa upisa podataka.

Uprkos efikasnom upravljanju prostorom, tokom rada fajl sistema dolazi do pojave fragmentacije. Fragmentacija predstavlja situaciju u kojoj podaci jednog fajla nisu smešteni u uzastopnim klasterima, već su rasuti po različitim delovima diska. Na taj način otežava se sekvencijalno čitanje podataka i može doći do smanjenja performansi sistema.

Postoje dva osnovna tipa fragmentacije: interna i eksterna fragmentacija. Interna fragmentacija nastaje kada dodeljeni klaster nije u potpunosti iskorišćen, pa deo prostora unutar klastera ostaje neiskorišćen. Ova pojava je posebno izražena kod fajlova čija veličina nije deljiva sa veličinom klastera, usled čega poslednji klaster fajla ostaje delimično prazan.

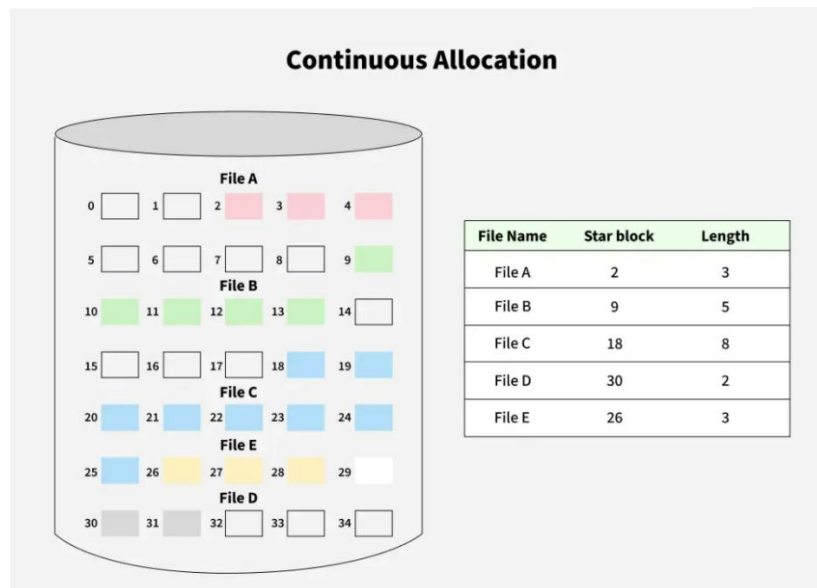
Eksterna fragmentacija nastaje kada se slobodan prostor na disku rasporedi u više manjih nepovezanih celina, što otežava dodelu kontinualnog prostora za nove fajlove. Ovaj problem je karakterističan za fajl sisteme koji koriste kontinuiranu alokaciju i može dovesti do većeg broja disk operacija i sporijeg pristupa podacima.

Sa aspekta digitalne forenzike, posebno su značajni nealocirani prostor i *slack space*. Nealocirani prostor predstavlja delove diska koji trenutno nisu dodeljeni nijednom fajlu, dok *slack space* označava neiskorišćen deo poslednjeg klastera fajla. Ove oblasti mogu sadržati ostatke prethodno obrisanih ili izmenjenih podataka, zbog čega predstavljaju važan izvor digitalnih tragova u forenzičkim istragama.

3.4.2. Metode alokacije prostora

3.4.2.1. Kontinuirana alokacija

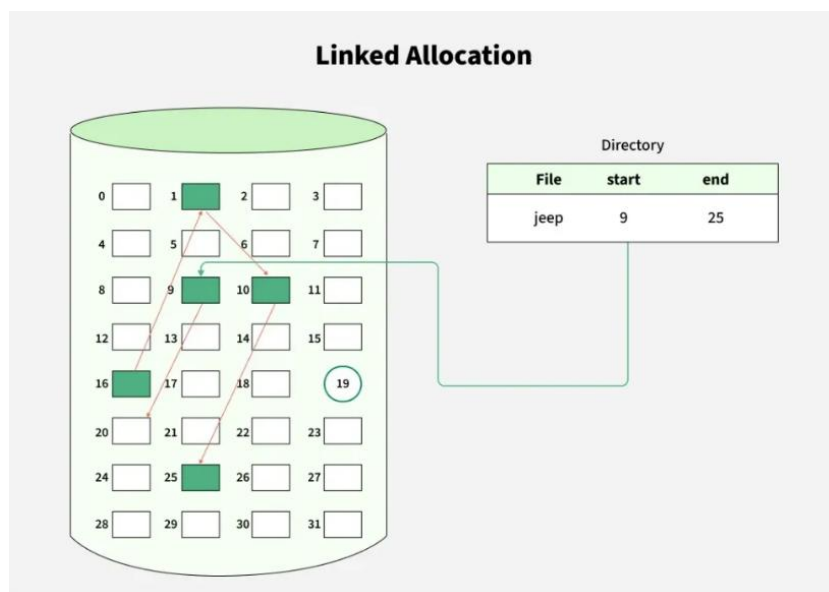
Fajl zauzima uzastopne klaster na disku i kod ove metode omogućen je brz pristup podacima, ukoliko su poznati početni blok i dužina fajla izražena brojem blokova. Ovakav pristup može dovesti do spoljne fragmentacije i otežanog pronalaženja većih blokova slobodnog prostora. Direktorijumski zapis sadrži adresu početnog bloka i dužinu dodeljenog prostora.



Slika 1. Primer kontinuirane alokacije [2]

3.4.2.2. Lančana alokacija

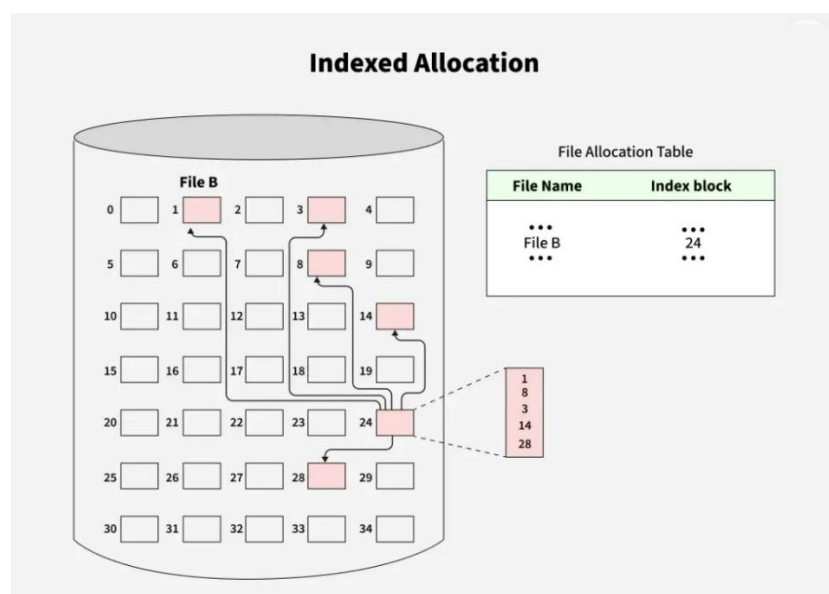
Fajl je organizovan kao povezana lista blokova koji ne moraju da budu smešteni na susednim lokacijama diska. Svaki klaster sadrži pokazivač na sledeći klaster u fajlu. Ova metoda eliminiše problem kontinuiteta, ali može usporiti pristup podacima zbog potrebe za praćenjem lanaca. Direktorijumski zapis sadrži pokazivače na prvi i poslednji blok fajla, dok svaki blok sadrži pokazivač na naredni blok.



Slika 2. Primer lančane alokacije [2]

3.4.2.3. Indeksirana alokacija

Koristi poseban blok koji se naziva indeksni blok koji čuva liste svih klastera koji pripadaju jednom fajlu. Svaki fajl mora da poseduje sopstveni indeksni blok, dok direktorijumski zapis sadrži adresu tog indeksnog bloka. Ovaj pristup omogućava direktan pristup podacima i efikasnije upravljanje većim fajlovima.



Slika 3. Primer indeksirane alokacije [2]

Svaka od ovih metoda ima svoje prednosti i mane koje su predstavljene u tabeli 1.

Metoda	Prednosti	Nedostaci
Kontinuirana alokacije	1. Podržava sekvencijalni i direktni pristup podacima. 2. Veoma brz pristup jer su svi blokovi smešteni jedan pored drugog. 3. Zahteva mali broj pomeranja glave diska.	1. Dovodi do interne i eksterne fragmentacije. 2. Proširivanje fajla je otežano jer zahteva postojanje slobodnog kontinualnog prostora na disku.
Lančana alokacija	1. Omogućava jednostavno povećavanje veličine fajla. 2. Ne pati od eksterne fragmentacije. 3. Efikasnije koristi prostor na disku od kontinuirane alokacije.	1. Pristup podacima je sporiji jer je potrebno pratiti lanac pokazivača između blokova. 2. Ne podržava direktan (nasumičan) pristup podacima. 3. Dodatni prostor se troši na čuvanje pokazivača u svakom bloku.
Indeksirana alokacija	1. Omogućava brz direktan pristup svim blokovima fajla. 2. Eliminise problem eksterne fragmentacije. 3. Pogodna je za rad sa velikim fajlovima.	1. Zahteva dodatni prostor za indeksni blok. 2. Kod veoma malih fajlova zauzimanje čitavog indeksnog bloka može predstavljati neefikasno korišćenje memorije.

Tabela 1. Prednosti i mane metoda alokacije slobodnog prostora [2]

U praktičnim implementacijama fajl sistema često se koriste kombinovane metode alokacije koje optimizuju performanse i korišćenje prostora.

3.4.3. Šeme indeksiranja

Kod veoma velikih fajlova jedan indeksni blok često nije dovoljan za čuvanje svih pokazivača na blokove podataka. Zbog toga se koriste naprednije šeme indeksiranja kao što su *linked scheme*, *multilevel index* i *inode* struktura.

3.4.3.1. Linked scheme

U ovoj šemi više indeksnih blokova povezano je u lanac. Svaki indeksni blok sadrži određeni broj pokazivača na blokove podataka i pokazivač na sledeći indeksni blok. Lanac se nastavlja sve dok se ne obuhvate svi blokovi fajla.

3.4.3.2. Multilevel Index

Umesto jednog indeksnog bloka koristi se hijerarhija indeksnih blokova. Indeksni blok prvog nivoa pokazuje na indeksne blokove drugog nivoa, dok oni sadrže adrese blokova podataka. Po potrebi se mogu koristiti i dodatni nivoi indeksiranja.

3.4.3.3. Inode struktura

Savremeni fajl sistemi najčešće koriste kombinovanu šemu zasnovanu na *inode* strukturi. *Inode* predstavlja posebnu strukturu koja sadrži metapodatke o fajlu i adrese blokova podataka. *Inode* obično sadrži:

- Direktne pokazivače koji neposredno ukazuju na blokove podataka.
- Jednostruko indirektni pokazivač, koji pokazuje na blok sa dodatnim adresama blokova podataka.
- Dvostruko indirektni pokazivač, koji uvodi još jedan nivo posredovanja.
- Trostruko indirektni pokazivač, namenjen radu sa izuzetno velikim fajlovima.

4. Najpoznatiji fajl sistemi

4.1. FAT (File Allocation Table)

FAT je jedan od najstarijih i najjednostavnijih fajl sistema. Prvobitno je razvijen za MS-DOS, ali se još uvek koristi na mnogim uređajima kao što su USB fleš memorije, SD kartice i IoT uređaji. Dve glavne verzije ovog sistema su FAT16 i FAT32. FAT16 je predstavljen 1987. godine uz DOS 3.31, dok je FAT32 uveden sa Windows 95 OSR2 (MS-DOS 7.1) 1996. godine [3].

Osnovu FAT sistema čini tabela alokacije fajlova (File Allocation Table), koja sadrži informacije o rasporedu klastera na disku. Za svaki klaster postoji odgovarajući zapis u tabeli koji označava da li je klaster slobodan, zauzet, predstavlja kraj fajla ili pokazuje na naredni klaster koji pripada istom fajlu. Na taj način operativni sistem može da prati raspored podataka i rekonstruiše sadržaj fajla.

Pored FAT tabele, značajnu ulogu ima i direktorijumski zapis, koji sadrži osnovne informacije o fajlu, kao što su ime, veličina, vremenske oznake i broj početnog klastera. Prilikom pristupa fajlu, operativni sistem prvo pronalazi njegov početni klaster u direktorijumskom zapisu, a zatim prati lanac pokazivača u FAT tabeli sve do oznake kraja fajla.

Sa aspekta digitalne forenzike, FAT je posebno interesantan zbog načina na koji vrši brisanje fajlova. Kada korisnik obriše fajl, podaci se fizički ne uklanjaju sa diska. Prvi karakter imena fajla u direktorijumskom zapisu menja se posebnom oznakom (0xE5), čime se prostor označava kao slobodan za budući upis. Sve dok novi podaci ne prepišu te klastere, sadržaj obrisano fajla ostaje prisutan na disku i može biti oporavljen forenzičkim alatima. Zbog toga FAT fajl sistemi često predstavljaju značajan izvor digitalnih dokaza tokom forenzičkih istraga.

Prednosti i nedostaci FAT fajl sistema prikazani su u tabeli 2.

Prednosti	Nedostaci
Jednostavna struktura FAT fajl sistema olakšava implementaciju i korišćenje, što ga čini pogodnim za uređaje sa ograničenim resursima.	Podložan je fragmentaciji, pri čemu se delovi fajla mogu nalaziti na različitim lokacijama na disku, što može smanjiti performanse sistema.
Zbog jednostavne strukture, fajlovi u sistemu se relativno lako mogu obnoviti u slučaju oštećenja podataka ili slučajnog brisanja.	Ne poseduje napredne funkcionalnosti prisutne kod savremenih fajl sistema, kao što su dozvole pristupa, <i>journaling</i> , enkripcija i kompresija podataka.
Podržan je od strane većine operativnih sistema, uključujući Windows, Linux i	Fajlovi na FAT16 sistemu ne mogu imati ime duže od 8.3 karaktera, što znači 8 karaktera za ime i 3 za ekstenziju fajla.

macOS, bez potrebe za dodatnim softverom.	
Omogućava relativno jednostavnu konverziju u druge fajl sisteme, poput NTFS-a, bez gubitka podataka.	Obezbeđuje veoma ograničene bezbednosne mehanizme, pa ne pruža adekvatnu zaštitu podataka od neovlašćenih izmena.
Široko je zastupljen na USB memorijama, memorijskim karticama i drugim prenosivim uređajima za skladištenje podataka.	Ne poseduje mehanizme otpornosti na greške i oporavka nakon pada sistema, kakve imaju moderniji fajl sistemi.
Podržava veliki broj različitih uređaja i platformi zahvaljujući visokoj kompatibilnosti.	

Tabela 2. Prednosti i nedostaci FAT fajl sistema [3, 4]

4.2. exFAT (Extended File Allocation Table)

exFAT je fajl sistem koji je predstavio Microsoft 2006. godine kao poboljšanu verziju FAT32 fajl sistema u okviru Windows CE 6.0. Rešava neka od ograničenja FAT32, omogućavajući veće veličine fajlova i bolje performanse. Koristi se za uređaje poput eksternih SSD diskova, hard diskova i SD kartica, jer pruža kompatibilnost sa više operativnih sistema [3].

exFAT zadržava osnovni koncept FAT porodice fajl sistema zasnovan na tabeli alokacije, ali uvodi poboljšanja namenjena radu sa velikim kapacitetima skladištenja i velikim fajlovima. Fajl sistem koristi unapređene strukture za upravljanje slobodnim prostorom što omogućava efikasnije pronalaženje slobodnih klastera i smanjuje potrebu za učestalim pretraživanjem cele tabele alokacije. Zbog toga je posebno pogodan za fleš memorije, SDXC kartice i eksterne uređaje za skladištenje podataka.

Sa aspekta digitalne forenzike, exFAT zadržava određene karakteristike FAT sistema koje mogu biti korisne tokom istrage. Prilikom brisanja fajlova podaci se ne uklanjaju odmah sa medijuma, već se prostor samo označava kao slobodan za budući upis. To omogućava oporavak obrisanih podataka sve dok ne budu prepisani novim sadržajem. Zbog izostanka *journaling* mehanizma, exFAT pruža manje istorijskih informacija o aktivnostima na sistemu u poređenju sa fajl sistemima kao što je NTFS.

Prednosti i nedostaci FAT fajl sistema prikazani su u tabeli 3.

Prednosti	Nedostaci
exFAT može da rukuje fajlovima većim od 4GB, što ga čini pogodnim za skladištenje velikih multimedijalnih fajlova.	Bezbednost podataka nije pouzdana kao kod modernih fajl sistema.
Poboljšava iskorišćenje prostora na disku u poređenju sa starijim FAT fajl sistemima jer koristi manje veličine klastera, a time se smanjuje količina neiskorišćenog prostora kod manjih fajlova.	Kao i FAT fajl sistemi, exFAT je i dalje podložan fragmentaciji. Kako se fajlovi kreiraju, menjaju i brišu, može doći do fragmentacije, što vremenom dovodi do smanjenja performansi.
Podržan je od strane većine operativnih sistema, uključujući Windows, Linux i macOS, bez potrebe za dodatnim softverom.	exFAT nema napredne funkcije koje poseduju moderni fajl sistemi, kao što su dozvole na nivou fajlova, <i>journaling</i> ili enkripcija na nivou fajl sistema.
Podržava veoma velike particije i uređaje za skladištenje velikog kapaciteta.	Ne poseduje ugrađene mehanizme za oporavak nakon pada sistema ili nestanka napajanja.

Tabela 3. Prednosti i nedostaci exFAT fajl sistema [3]

4.3. NTFS (New Technology File System)

NTFS (New Technology File System) je podrazumevani fajl sistem za operativne sisteme zasnovane na Windows NT platformi, počevši od 1993. godine sa Windows NT 3.1, pa sve do Windows 11. Nudi napredne funkcije kao što su dozvole na fajlovima, enkripcija, kompresija i *journaling*. NTFS podržava velike veličine fajlova i particija, što ga čini pogodnim za savremene uređaje za skladištenje. Međutim, ima ograničenu kompatibilnost sa operativnim sistemima koji nisu Windows [3].

Za razliku od FAT sistema, NTFS ne koristi jednostavnu tabelu alokacije za praćenje fajlova, već se zasniva na centralnoj strukturi pod nazivom Master File Table (MFT). MFT sadrži zapise o svim fajlovima i direktorijumima na particiji, uključujući njihove metapodatke, lokacije podataka i attribute. Zahvaljujući ovakvoj organizaciji, NTFS omogućava efikasnije upravljanje podacima, veću pouzdanost i napredne funkcionalnosti.

Jedna od najvažnijih karakteristika NTFS fajl sistema jeste podrška za *journaling*. Pre izvršavanja promena u fajl sistemu, informacije o tim promenama beleže se u dnevnik transakcija. U slučaju nestanka napajanja ili pada sistema, dnevnik omogućava oporavak konzistentnog stanja fajl sistema bez potrebe za dugotrajnim proverama diska.

NTFS podržava napredne bezbednosne mehanizme, uključujući kontrolu pristupa na nivou fajlova i direktorijuma (ACL – Access Control List), enkripciju podataka, kompresiju fajlova i

kvote za korišćenje prostora na disku. Takođe podržava veoma velike fajlove i particije, kao i Unicode nazive fajlova dužine do 255 karaktera.

Sa aspekta digitalne forenzike, NTFS predstavlja izuzetno bogat izvor informacija. Pored samih fajlova, značajne forenzičke artefakte predstavljaju Master File Table (MFT), Journal (\$LogFile), USN Journal, informacije o obrisanim fajlovima i različiti metapodaci koji omogućavaju rekonstrukciju aktivnosti korisnika i vremenske linije događaja.

Prednosti i nedostaci NTFS fajl sistema prikazani su u tabeli 4.

Prednosti	Nedostaci
Pruža snažan model bezbednosti sa dozvolama na nivou fajla. Omogućava postavljanje dozvola za pojedinačne fajlove i direktorijume, kontrolišući prava pristupa za korisnike i grupe.	Manja kompatibilnost sa starijim operativnim sistemima i uređajima.
Podržava kompresiju podataka, kvote i <i>alternate data streams</i> .	Vremenom NTFS fajl sistem može postati fragmentisan, posebno kada se fajlovi kreiraju, modifikuju i brišu. Fragmentacija može dovesti do smanjenih performansi jer sistem mora pristupati rasutim delovima fajlova.
Podržava veoma velike fajlove i particije.	Složenija analiza i oporavak u odnosu na jednostavnije fajl sisteme.

Tabela 4. Prednosti i nedostaci NTFS fajl sistema [3]

4.4. HFS (Hierarchical File System)

HFS (Hierarchical File System) je fajl sistem prvobitno razvijen od strane kompanije Apple za korišćenje na Macintosh računarima, kao zamena za MFS (Macintosh File System). Iako je HFS imao značajnu ulogu u razvoju Mac OS ekosistema, vremenom su se pojavila ograničenja u pogledu skalabilnosti, fragmentacije i nedostatka naprednih funkcionalnosti poput *journaling*-a. Zbog toga je kasnije zamenjen unapređenim HFS+ fajl sistemom, a zatim i APFS-om.

HFS je uveo koncept direktorijumske strukture sa folderima i podfolderima, što je predstavljalo značajno unapređenje u odnosu na dotadašnji ravni model organizacije podataka.

Razvoj HFS-a bio je direktno uslovljen porastom kapaciteta skladišnih uređaja, jer su se floppy diskovi zamenjivali hard diskovima većih kapaciteta. Kako bi omogućio upravljanje većim količinama podataka, HFS uvodi strukturu zasnovanu na *allocation* blokovima, kao i posebne sistemske strukture za praćenje lokacije fajlova na disku, poput Catalog File i Extents Overflow File, koje su implementirane kao B-tree strukture.

Jedna od specifičnosti HFS fajl sistema jeste podrška za *dual-fork* model fajlova, koji se sastoji od *data fork* i *resource fork*. *Data fork* sadrži primarne podatke fajla, dok *resource fork* čuva dodatne strukturalne i interfejsne informacije, kao što su ikonice, dijalozi i metapodaci aplikacija.

Sa aspekta digitalne forenzike, HFS predstavlja fajl sistem sa ograničenim, ali i dalje značajnim forenzičkim potencijalom. Zbog načina na koji organizuje podatke kroz B-tree strukture, moguće je rekonstruisati osnovne informacije o strukturi fajlova i direktorijuma na disku. Međutim, u poređenju sa modernim fajl sistemima, HFS ne čuva detaljne istorijske zapise o aktivnostima korisnika. Brisanje fajlova u HFS sistemu ne podrazumeva trenutno uklanjanje podataka sa medijuma, već se prostor označava kao slobodan za ponovno korišćenje. Ovo omogućava delimičan oporavak podataka, sve dok isti sektor nije prepisan novim informacijama. Ipak, zbog nepostojanja *journaling* mehanizma i ograničenih metapodataka, forenzička analiza je često otežana i manje precizna u poređenju sa savremenim fajl sistemima [5].

Prednosti i nedostaci HFS fajl sistema prikazani su u tabeli 5.

Prednosti	Nedostaci
HFS ima relativno jednostavnu i lako razumljivu strukturu, što olakšava njegovu implementaciju i upotrebu.	HFS ne podržava mnoge savremene funkcije, kao što su dozvole na nivou fajla, <i>journaling</i> , kompresiju fajlova ili enkripciju.
Omogućava hijerarhijsku organizaciju fajlova kroz direktorijume i poddirektorijume.	HFS podržava fajlove do 2 GB veličine i particije do 2 TB.
Podržava <i>dual-fork</i> strukturu fajlova (data i resource fork).	Kopiranje sa HFS na druge fajl sisteme, često dovodi do gubljenja <i>resource fork</i> -a, zbog čega fajlovi mogu postati neupotrebljivi ako se kasnije ponovo prebace na HFS.
Integracija sa klasičnim Mac OS Finder okruženjem i metapodacima.	Slaba efikasnost na velikim diskovima zbog fragmentacije i velikih <i>allocation</i> blokova.
Pogodan za rad sa starijim Mac sistemima i arhivskim podacima.	Ograničena skalabilnost u odnosu na moderne fajl sisteme.

Tabela 5. Prednosti i nedostaci HFS fajl sistema [5]

4.5. APFS (Apple File System)

APFS (Apple File System) je fajl sistem koji je razvila kompanija Apple za macOS, iOS i druge Apple uređaje. Predstavljen je sa macOS 10.13 2017. godine kao zamena za stariji HFS+ i od tada predstavlja podrazumevani fajl sistem na Apple uređajima. Specijalno je dizajniran tako da optimizuje performanse i kompatibilnost sa Apple hardverom i softverom. APFS uključuje funkcije kao što su kloniranje, enkripcija na nivou fajla i poboljšane performanse na SSD diskovima [3].

Za razliku od tradicionalnih fajl sistema, APFS koristi koncept kontejnera i volumena koji dele isti fizički prostor na disku. Ovakva arhitektura omogućava fleksibilnije upravljanje skladištem i smanjuje ograničenja klasične particione strukture.

APFS uvodi mehanizam *snapshots*, koji omogućava čuvanje konzistentnog stanja fajl sistema u određenom vremenskom trenutku. Ovo omogućava analizu istorijskih stanja sistema i potencijalno vraćanje podataka koji su kasnije izmenjeni ili obrisani.

Takođe, APFS podržava nativnu enkripciju, kao i efikasno upravljanje prostorom kroz deljenje kapaciteta između više volumena. Ove karakteristike povećavaju sigurnost i efikasnost, ali uvode dodatnu složenost u strukturu fajl sistema.

Sa aspekta digitalne forenzike, APFS je značajan zbog mogućnosti očuvanja istorijskih stanja sistema kroz *snapshot*-e, ali i izazovan zbog enkripcije i složenog načina organizacije podataka [6].

Prednosti i nedostaci APFS fajl sistema prikazani su u tabeli 6.

Prednosti	Nedostaci
Optimizovan je za moderne SSD i fleš memorije, što rezultira visokim performansama.	Ograničena kompatibilnost jer je vezan za Apple ekosistem.
Omogućava efikasno upravljanje prostorom kroz koncepte kontejnera i volumena.	Kompleksna interna struktura otežava analizu i forenzičku obradu.
Snapshot mehanizam omogućava očuvanje stanja fajl sistema kroz vreme.	Snapshot mehanizam povećava složenost skladištenja i analize podataka.
Podržava nativnu enkripciju na nivou fajl sistema.	Pristup podacima može biti otežan bez enkripcijskih ključeva.
Povećava pouzdanost i konzistentnost podataka u radu sistema.	Zahteva specijalizovane alate za naprednu forenzičku analizu.

Tabela 6. Prednosti i nedostaci APFS fajl sistema [3, 6]

4.6. Ext4 (Fourth Extended File System)

Ext4 (Fourth Extended File System) je najzastupljeniji fajl sistem u Linux operativnim sistemima. On je naslednik Ext3 i nudi brojna poboljšanja u performansama, skalabilnosti i pouzdanosti. Ext4 je podrazumevani fajl sistem za mnoge Linux distribucije [3].

Ext4 podržava rad sa veoma velikim fajlovima i particijama, što ga čini pogodnim za savremene sisteme koji obrađuju velike količine podataka. Za organizaciju podataka koristi *extents* model, pri kojem se fajlovi predstavljaju kao skupovi kontinualnih blokova, što smanjuje fragmentaciju i poboljšava performanse u odnosu na ranije EXT verzije.

Jedna od važnih karakteristika EXT4 fajl sistema jeste *journaling* mehanizam sa kontrolnim sumama, koji omogućava praćenje promena i povećava integritet podataka. U slučaju sistemskog pada ili prekida napajanja, *journaling* omogućava brži i pouzdaniji oporavak konzistentnog stanja fajl sistema.

Sa aspekta digitalne forenzike, EXT4 omogućava analizu strukture fajl sistema, uključujući metapodatke poput vremena pristupa, modifikacije, dozvola i hijerarhije direktorijuma. Posebno značajan forenzički aspekt predstavlja *journal*, koji može sadržati informacije o nedavno izvršenim operacijama, čime se može rekonstruisati vremenska linija aktivnosti na sistemu [7].

Prednosti i nedostaci EXT4 fajl sistema prikazani su u tabeli 7.

Prednosti	Nedostaci
Obezbeđuje smanjen rizik od oštećenja podataka i obezbeđuje konzistentnost fajl sistema.	Ograničena kompatibilnost sa operativnim sistemima koji nisu Linux.
Ext4 podržava znatno veće fajlove i particije i omogućava efikasno upravljanje njima.	Enkripcija na Android uređajima može otežati pristup podacima.
Smanjena fragmentacija zahvaljujući <i>extents</i> modelu.	Broj i veličina inoda se fiksiraju prilikom kreiranja sistema i ne mogu se dinamički menjati bez reformatiranja diska.
<i>Journaling</i> sa kontrolnim sumama povećava integritet podataka.	Ext4 ne podržava transparentnu kompresiju podataka, što znači da ne može uštedeti prostor na disku automatskim smanjivanjem veličine fajlova.

Tabela 7. Prednosti i nedostaci Ext4 fajl sistema [3, 7]

5. Najčešće tehnike za skrivanje podataka u okviru fajl sistema

Skrivanje podataka u okviru fajl sistema predstavlja jednu od često korišćenih metoda za namensko prikrivanje informacija. Za razliku od kriptografije koja čini podatke nečitljivim, tehnike skrivanja u fajl sistemu često čine podatke potpuno nevidljivim za prosečnog korisnika i čak za većinu standardnih softverskih alata. Razumevanje ovih tehnika je od suštinskog značaja kako za forenzičare tako i za IT bezbednosne stručnjake.

5.1. Hidden atribut fajlova

Najjednostavnija metoda skrivanja informacija koju neko može da iskoristiti jeste skrivanje fajlova ili direktorijuma putem dijaloga za podešavanje svojstva u Windows operativnim sistemima. Pošto je ovo najjednostavniji način skrivanja informacija on je i vrlo lako uočljiv. Da bi se otkrili skriveni fajlovi ili direktorijumi, dovoljno je samo uključiti prikaz skrivenih fajlova i direktorijuma u podešavanjima pregleda direktorijuma. Zbog toga je efikasnost ove metode za prikrivanje podataka u kriminalne svrhe veoma ograničena.

5.2. Skrivanje podataka u nealociranom prostoru

Nealocirani prostor predstavlja sve blokove na disku koji nisu trenutno dodeljeni nijednom fajlu. Ovaj prostor nastaje kada se fajlovi brišu, umesto fizičkog brisanja operativni sistemi samo označavaju blokove kao dostupne za pisanje, ali fizički podaci ostaju na disku.

Operativni sistemi čine nealocirani prostor nevidljivim za standardne korisnike. Direktorijumi, tabele alokacije i druge strukture fajl sistema ne referenciraju nealocirani prostor. Osim ako korisnik ne koristi specijalne forenzičke alate, on nikada neće biti svestan postojanja ovog prostora što ga čini idealnim mestom za skrivanje podataka.

Veličina nealociranog prostora varira zavisno od toga kako se disk koristi. Aktivno korišćeni sistemi sa čestim operacijama pisanja i brisanja mogu imati značajnu količinu nealociranog prostora.

Najjednostavnija metoda je direktno upisivanje podataka u nealocirani prostor koristeći alate koji omogućavaju pristup na niskom nivou. Ova metoda ima relativno mali rizik jer se nealocirani blokovi obično ne koriste ako disk nije prepun.

Druga metoda zasniva se na korišćenju obrisanih fajlova za skrivanje podataka. Korisnik najpre kreira fajl i u njega smešta informacije koje želi da sakrije. Nakon toga fajl briše, čime se njegov zapis uklanja iz struktura fajl sistema, dok sadržaj ostaje sačuvan u odgovarajućim klasterima na disku. Sve dok ti klasteri ne budu prepisani novim podacima, informacije je moguće oporaviti. Na taj način podaci postaju nevidljivi za standardne mehanizme pregleda fajlova, ali i dalje fizički postoje u nealociranom prostoru diska.

Forenzičke tehnike poznate kao *carving* pokušavaju da rekonstruišu fajlove iz nealociranog prostora pronalaženjem karakterističnih zaglavlja i završetaka fajlova. Na primer, JPEG fajlovi se mogu detektovati pronalaženjem karakterističnog heksadecimalnog niza FF D8 FF koji svi JPEG fajlovi moraju da sadrže na početku.

5.3. Alternate Data Streams

Alternate Data Streams (ADS) predstavljaju NTFS specifičnu mogućnost koja dozvoljava da jedan fajl sadrži više od jednog toka podataka. Svaki fajl u NTFS sistemu ima glavni tok podataka koji se koristi za čuvanje glavnog sadržaja fajla, ali NTFS dozvoljava kreiranje dodatnih tokova koji mogu sadržavati bilo kakve podatke. Ova mogućnost je originalno predviđena za kompatibilnost sa Mac sistemima, koji koriste *fork* strukturu i za čuvanje specifičnih atributa fajla, ali je brzo shvaćeno da može da se iskoristi kao moćan mehanizam za skrivanje podataka.

U NTFS sistemu, svaki fajl je zapravo kolekcija atributa. Glavni atribut je \$DATA koji sadrži glavni sadržaj fajla, a alternativni tokovi su dodatni \$DATA atributi sa različitim imenima koji se ne prikazuju u normalnom pregledu fajla. Dodatno je zgodno što se veličina ADS-a ne prikazuje u svojstvima fajla kod većine standardnih alata, kao što je Windows Explorer. Kada korisnik pogleda svojstva fajla, veličina fajla će biti prikazana kao originalna odnosno veličina glavnog toka, a ADS će biti potpuno nevidljiv.

Posebno opasna karakteristika ADS-a je što može da sadrži izvršni kod. Ako ADS sadrži Windows izvršnu datoteku (.exe), ona se može pokrenuti direktno iz ADS-a, pa tako operativni sistem onda može da pokrene izvršenje trojanskog fajla, čineći ga potpuno nevidljivim većini antivirus programa koji skeniraju samo glavne tokove fajlova.

Korišćenje opcija *-R (recurse)* ili *-S (summary)* sa *dir* komandom može otkriti ADS-ove, a postoje i specijalizovani forenzički alati koji imaju ugrađenu mogućnost za detektovanje i analizu ADS-a. Takođe moguće je analizirati NTFS MFT (Master File Table) strukture i pronaći sve attribute fajla, jer MFT sadrži sve informacije o svim tokovima, uključujući ADS-ove.

5.4. Steganografija na nivou fajl sistema

Steganografski fajl sistemi predstavljaju napredniji pristup skrivanju podataka čiji cilj nije samo sakrivanje sadržaja fajla, već prikrivanje samog postojanja podataka. Za razliku od klasičnih metoda, kao što su skriveni atributi ili *slack space*, gde se koristi postojeći fajl sistem, steganografski fajl sistemi su posebno dizajnirani da omoguće tajno skladištenje podataka uz očuvanje izgleda normalnog sistema za skladištenje.

Osnovna ideja ovih sistema jeste da se skriveni podaci rasporede po prostoru za skladištenje na način koji čini da ih je nemoguće razlikovati od slučajnih podataka ili uobičajenih ostataka fajl sistema. Lokacije na kojima se podaci čuvaju često se određuju pomoću pseudorandom

funkcija koje zavise od tajnog ključa ili lozinke korisnika. Bez poznavanja ključa, analiza diska ne može jednostavno da utvrdi koji blokovi pripadaju skrivenim fajlovima.

Jedan od poznatijih primera praktične implementacije jeste StegFS, koji proširuje standardni ext fajl sistem i omogućava čuvanje skrivenih fajlova u prostoru koji izgleda kao običan slobodan prostor. Skriveni sadržaj se šifrjuje i raspoređuje na više lokacija kako bi se otežala njegova detekcija.

Za razliku od jednostavnijih tehnika skrivanja, steganografski fajl sistemi predstavljaju kompletnu arhitekturu za tajno skladištenje podataka čija je prednost visok nivo prikrivanja, ali ovakav nivo zaštite zahteva složeniju implementaciju, dodatne operacije prilikom čitanja i upisa podataka i često dovodi do smanjenja performansi sistema.

5.5. Skrivanje podataka u metapodacima fajl sistema

Metapodaci fajl sistema predstavljaju informacije koje operativni sistem koristi za upravljanje fajlovima, kao što su ime fajla, veličina, vremenske oznake, dozvole pristupa, lokacija fizičkih blokova i drugi atributi. Pored podataka koje korisnik direktno vidi, fajl sistemi često sadrže dodatne strukture i rezervisana polja koja mogu predstavljati potencijalno mesto za skrivanje informacija.

Kod različitih fajl sistema postoje različite strukture koje čuvaju metapodatke. Na primer, NTFS koristi Master File Table (MFT) zapise, ext sistemi koriste inode strukture, dok FAT fajl sistemi koriste direktorijumske zapise i FAT tabele za praćenje zauzetosti prostora. Ukoliko određeni delovi ovih struktura nisu iskorišćeni ili nisu proveravani tokom standardnog rada, mogu se zloupotребiti za smeštanje malih količina skrivenih podataka.

Jedan od načina skrivanja jeste korišćenje neiskorišćenih ili rezervisanih polja u strukturama fajl sistema. Podaci upisani na ovaj način ne predstavljaju zaseban fajl i nisu dostupni kroz standardne korisničke interfejse. Međutim, osoba koja poznaje unutrašnju organizaciju fajl sistema može ih pronaći analizom odgovarajućih struktura.

Pored direktnog smeštanja podataka u metapodatke, moguće je koristiti i manipulaciju postojećim atributima fajlova. Na primer, promena određenih atributa ili umetanje dodatnih informacija u strukture koje opisuju fajl može omogućiti skrivanje podataka bez kreiranja novih objekata u fajl sistemu.

Prednost ove metode je što skriveni sadržaj može ostati relativno neprimetan jer se nalazi u delovima sistema koji nisu namenjeni direktnom čuvanju korisničkih podataka. Međutim, kapacitet je uglavnom veoma ograničen, jer su metapodaci mali i njihova promena može narušiti integritet fajl sistema.

Sa forenzičkog stanovišta, otkrivanje ovakvog načina skrivanja zahteva detaljnu analizu unutrašnjih struktura fajl sistema. Alati koji analiziraju samo vidljive fajlove i direktorijume uglavnom neće otkriti ovakav sadržaj, dok napredniji forenzički alati mogu pregledati sirove strukture diska i identifikovati nepravilnosti u metapodacima.

5.6. Skrivanje podataka u slack space-u

Slack prostor predstavlja deo prostora poslednjeg klastera koji je dodeljen fajlu, ali koji nije iskorišćen za čuvanje njegovog stvarnog sadržaja. Fajl sistemi uglavnom alociraju prostor u jedinicama fiksne veličine koje se nazivaju klasteri ili blokovi. Pošto veličina fajla najčešće nije tačan umnožak veličine klastera, poslednji dodeljeni klaster ostaje delimično popunjen, dok ostatak prostora ostaje neiskorišćen.

Na primer, ukoliko fajl sistem koristi klastere veličine 4096 bajtova, a fajl zauzima 3000 bajtova, operativni sistem će fajlu dodeliti ceo klaster, iako će poslednjih 1096 bajtova ostati neiskorišćeno.

Ovaj prostor nije vidljiv korisniku kroz standardne operacije rada sa fajlovima, jer fajl sistem vodi evidenciju samo o logičkoj veličini fajla, odnosno broju bajtova koji predstavljaju njegov stvarni sadržaj. Međutim, fizički gledano, preostali deo poslednjeg klastera i dalje postoji na medijumu i može biti iskorišćen za čuvanje dodatnih podataka.

Skrivanje podataka u *slack* prostoru zasniva se na direktnom pristupu blokovima uređaja i upisivanju dodatnog sadržaja iza kraja legitimnog sadržaja fajla. Pošto se veličina fajla i njegovi metapodaci ne menjaju, skriveni podaci ostaju neprimetni za većinu standardnih alata za pregled fajlova.

Ova tehnika predstavlja primer skrivanja podataka na nivou fizičke organizacije fajl sistema, jer se podaci ne dodaju kao novi fajl niti se menjaju postojeće strukture direktorijuma. Zbog toga tradicionalni korisnički programi ne mogu da ih prikažu, dok specijalizovani forenzički alati koji analiziraju sadržaj klastera mogu otkriti njihovo postojanje.

Sa forenzičkog aspekta, analiza *slack* prostora predstavlja važan deo istrage, jer se u njemu mogu pronaći ostaci prethodno korišćenih podataka ili namerno sakrivene informacije. Forenzički alati mogu izračunati očekivanu količinu *slack* prostora svakog fajla i analizirati sadržaj bajtova koji se nalaze iza logičkog kraja fajla.

Glavno ograničenje ove metode jeste relativno mali kapacitet za skladištenje podataka. Količina dostupnog prostora zavisi od veličine klastera i rasporeda fajlova na disku. Veći klasteri omogućavaju više prostora za skrivanje, ali istovremeno povećavaju gubitak prostora kod normalnog rada fajl sistema.

Zbog jednostavnosti implementacije i mogućnosti skrivanja podataka bez menjanja strukture fajl sistema, *slack space* predstavlja jednu od najčešće proučavanih tehnika skrivanja podataka u okviru fajl sistema.

5.7. Poređenje tehnika skrivanja podataka

5.7.1. Poređenje efikasnosti različitih tehnika skrivanja podataka

Efikasnost pojedinačnih tehnika skrivanja podataka zavisi od količine podataka koje je potrebno sakriti, nivoa bezbednosti koji se želi postići i verovatnoće forenzičke analize sistema.

Ne postoji univerzalna metoda koja je optimalna za sve scenarije, već se izbor tehnike prilagođava konkretnim zahtevima.

Kada je potrebno brzo sakriti manju količinu podataka, Alternate Data Streams (ADS) predstavljaju jedno od najpraktičnijih rešenja. Ova tehnika omogućava jednostavnu implementaciju i brz upis podataka bez vidljivih promena osnovnog fajla. Zbog toga je ADS pogodan za situacije u kojima je potrebno sakriti nekoliko kilobajta podataka uz minimalan uticaj na strukturu fajl sistema.

U slučajevima kada je potrebno dugoročno skladištiti veće količine skrivenih podataka, efikasnijim se pokazuje korišćenje *slack space*-a i nealociranog prostora. Ove oblasti diska mogu ponuditi veći raspoloživi kapacitet u odnosu na ADS, dok se dodatnom primenom šifrovanja povećava zaštita sadržaja od neovlašćenog pristupa. Kombinovanje više metoda istovremeno otežava forenzičku analizu i smanjuje verovatnoću uspešnog otkrivanja podataka.

Najviši nivo zaštite postiže se primenom integrisane steganografije, odnosno kombinovanjem više tehnika skrivanja podataka sa kriptografskim metodama. U takvom pristupu podaci mogu biti raspoređeni između ADS tokova, *slack space*-a i drugih skrivenih lokacija, dok se njihov sadržaj dodatno štiti šifrovanjem. Čak i ukoliko istražitelj uspe da pronađe deo skrivenih podataka, njihova interpretacija može biti izuzetno otežana bez poznavanja svih korišćenih metoda i odgovarajućih ključeva za dešifrovanje.

5.7.2. Napredni pristupi kombinovanju tehnika skrivanja podataka

Savremeni pristupi skrivanju podataka često se ne oslanjaju na jednu tehniku, već na kombinovanje više metoda koje se međusobno dopunjuju. Ovakav pristup poznat je kao slojevita zaštita i zasniva se na ideji da svaka dodatna tehnika predstavlja novu prepreku prilikom otkrivanja ili oporavka podataka.

U jednom mogućem scenariju, poruka se najpre može prikriti korišćenjem steganografskih metoda, nakon čega se dodatno šifrjuje radi zaštite njenog sadržaja. Ključ za dešifrovanje može biti sačuvan u ADS toku nekog fajla, dok se informacije o njegovoj lokaciji mogu nalaziti u *slack space*-u drugog fajla. Na taj način pojedinačni elementi sistema skrivanja postaju međusobno zavisni, pa pronalaženje jednog dela ne garantuje pristup kompletnim podacima.

Prednost slojevite zaštite ogleda se u činjenici da istražitelj mora da identifikuje i pravilno interpretira svaki nivo skrivanja kako bi došao do konačne informacije. Što je veći broj međusobno povezanih tehnika, to je proces analize složeniji i dugotrajniji.

Pored slojevitih pristupa, moguće je koristiti i polimorfne tehnike skrivanja podataka. Kod ovakvih metoda ne postoji unapred definisana lokacija ili način skrivanja informacija. Umesto toga, sistem može dinamički menjati tehniku koja se koristi prilikom svakog upisa podataka. U jednom slučaju podaci mogu biti smešteni u ADS tokove, u drugom u *slack space*, dok se u trećem može primeniti kombinacija više metoda zajedno sa šifrovanjem.

Ovakav pristup značajno otežava razvoj univerzalnih metoda za detekciju skrivenih podataka, jer se struktura i lokacija sakrivenih informacija menjaju od slučaja do slučaja. Zbog toga polimorfne tehnike predstavljaju poseban izazov za digitalnu forenziku i često zahtevaju detaljnu analizu celokupnog fajl sistema kako bi se identifikovali potencijalni tragovi skrivanja podataka.

6. Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

6.1. O alatu

U okviru ovog rada razvijen je Command-Line Interface (CLI) alat za skrivanje podataka u *slack* prostoru fajl sistema. Alat omogućava smeštanje proizvoljnih fajlova u neiskorišćeni prostor koji se nalazi na kraju poslednjeg alociranog bloka fajla, bez izmene sadržaja samog fajla.

Implementacija je realizovana u programskom jeziku Python i namenjena je radu na Linux operativnim sistemima sa ext4 fajl sistemom. Alat koristi sistemske komande i metapodatke fajl sistema za pronalaženje fizičkih lokacija blokova na disku, identifikaciju raspoloživog *slack* prostora i direktan pristup uređaju radi upisa i čitanja skrivenih podataka.

Osnovna ideja rada alata zasniva se na pronalaženju svih fajlova na ciljnoj particiji i određivanju njihovog *slack* prostora. Pronađeni *slack* prostor u fajl sistemu se deli na *slack* sektore podrazumevane veličine 512B, ali korisniku se ostavlja mogućnost odabira druge veličine. Ovi *slack* sektori se koriste za skladištenje podataka koje korisnik želi da sakrije. Da bi se omogućilo kasnije pronalaženje i rekonstrukcija skrivenih fajlova, implementirana je posebna struktura nazvana Slack Allocation Table (SAT) za praćenje alokacije sektora, čime se evidentira koji su fajlovi skriveni i gde se nalaze na disku. SAT tabela se perzistira u JSON formatu i čuva na eksternom medijumu ili drugoj lokaciji koju korisnik odredi. Prilikom ponovnog pokretanja alata korisnik prosleđuje lokaciju postojeće SAT tabele kako bi se obnovila evidencija prethodno skrivenih podataka.

6.2. Okruženje korišćenja alata i priprema uređaja

Iako je alat moguće primeniti na različitim uređajima za skladištenje podataka, osnovni scenario njegove upotrebe predviđa rad sa prenosivim USB memorijskim uređajem. Korišćenje zasebnog fizičkog uređaja omogućava bezbednije testiranje i primenu alata, jer se direktnim pristupom blokovima uređaja izbegava mogućnost slučajnog oštećenja podataka na sistemskom disku računara.

Pre korišćenja alata neophodno je pripremiti USB uređaj za rad. Ovaj postupak podrazumeva kreiranje particije, uklanjanje postojećeg sadržaja i formatiranje particije korišćenjem ext4 fajl sistema. Nakon toga particija se montira kako bi njen sadržaj bio dostupan operativnom sistemu.

Nakon povezivanja USB uređaja, operativni sistem mu dodeljuje odgovarajuću oznaku uređaja, na primer `/dev/sdb1`. Ova putanja omogućava direktan pristup particiji na nivou uređaja i koristi se za operacije koje zahtevaju čitanje i upis podataka na fizičkom medijumu. Istovremeno, montirana particija dobija i odgovarajuću *mount* putanju, na primer `/mnt/testusb`, preko koje se ostvaruje standardan pristup fajlovima i direktorijumima na uređaju.

Implementirani alat koristi obe putanje kao ulazne parametre. Putanja uređaja koristi se za direktan pristup sektorima i manipulaciju sadržajem *slack* prostora, dok se *mount* putanja koristi za analizu fajl sistema, pronalaženje fajlova i određivanje raspoloživog *slack* prostora.

6.3. Funkcionalnosti alata

Razvijeni alat podržava četiri osnovne funkcionalnosti:

6.3.1. Skrivanje fajlova

Prilikom skrivanja podataka korisnik navodi jedan ili više fajlova koje želi da sakrije smeštanjem u *slack* prostor. Izabrani fajlovi se upisuju u raspoložive sektore *slack* prostora i okviru SAT tabele se pamti njihova lokacija.

6.3.2. Čitanje skrivenih fajlova

Na osnovu identifikatora fajla i informacija sačuvanih u SAT tabeli moguće je pronaći *slack* sektore u kojima se nalaze skriveni podaci. Nakon čitanja sadržaja iz *slack* prostora, dobijaju se originalni fajlovi u okviru *output* direktorijuma koji je naveden kao ulazni parametar prilikom pokretanja.

6.3.3. Brisanje skrivenih fajlova

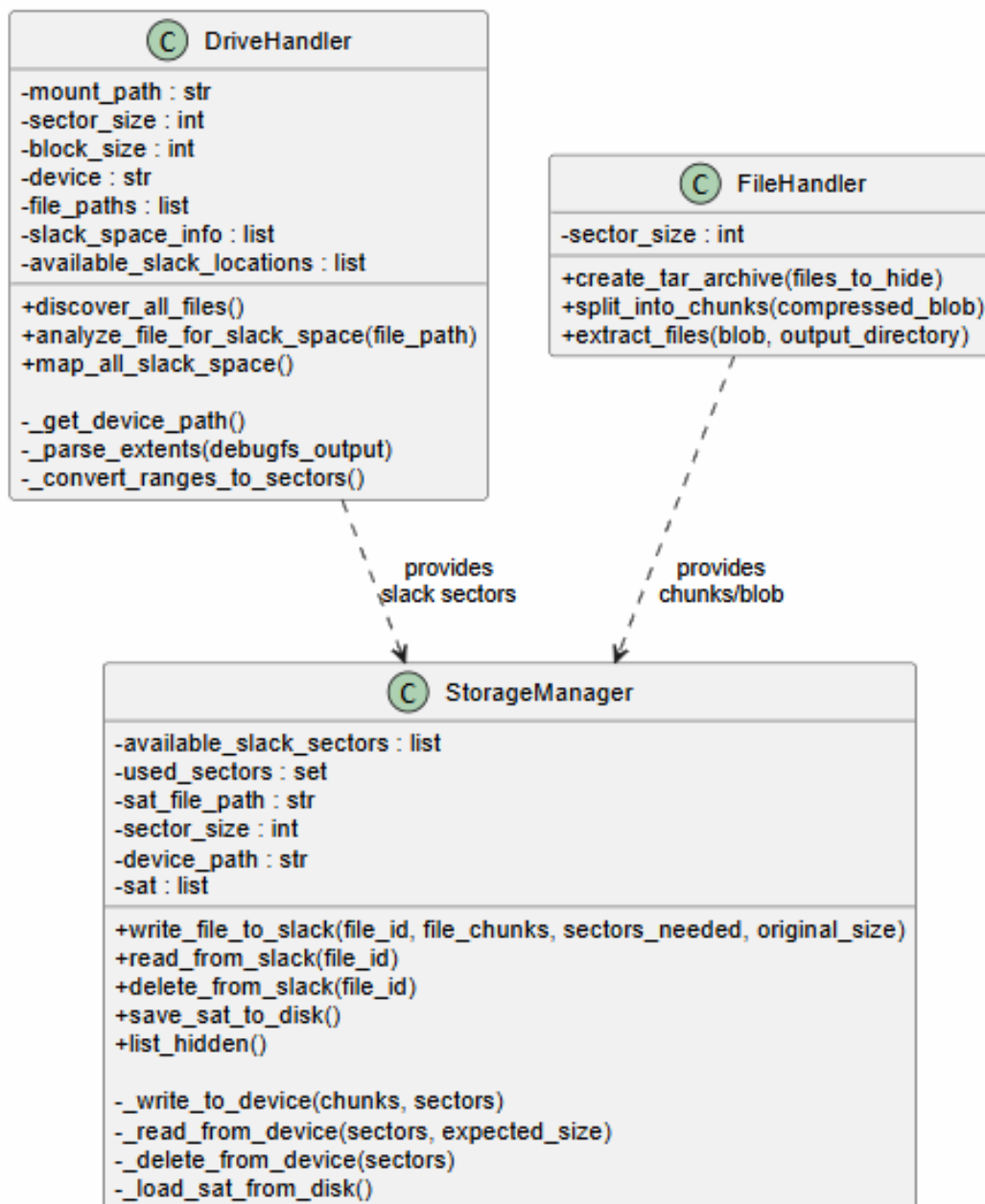
Alat omogućava bezbedno uklanjanje skrivenih podataka. Tokom procesa brisanja svi sektori koji pripadaju izabranom fajlu prepisuju se nulama, a odgovarajući zapis se uklanja iz SAT tabele. Na taj način se sprečava naknadna rekonstrukcija podataka jednostavnim čitanjem *slack* prostora.

6.3.4. Pregled skrivenih fajlova

Korisnik može dobiti listu svih trenutno skrivenih fajlova zajedno sa informacijama o njihovoj lokaciji, veličini, broju zauzetih sektora i vremenu smeštanja.

6.4. Arhitektura sistema

Arhitektura sistema organizovana je kroz tri osnovne klase *DriveHandler*, *FileHandler* i *StorageManager*. Implementirane su funkcije *read()*, *hide()*, *list_hidden_files()*, *delete()* koje pozivaju metode ovih klasa i na taj način omogućavaju funkcionalnosti sistema. Na slici 4 prikazan je UML klasni dijagram implementiranog rešenja.



Slika 4. UML dijagram

6.4.1. DriveHandler

Klasa *DriveHandler* zadužena je za interakciju sa fajl sistemom i pronalaženje raspoloživog *slack* prostora. Njene glavne odgovornosti su:

- određivanje putanje do uređaja,
- pronalaženje svih fajlova na particiji,
- analiza *inode* struktura i *extent* zapisa,
- izračunavanje veličine *slack* prostora,
- mapiranje *slack* prostora na fizičke sektore.

6.4.2. FileHandler

Klasa *FileHandler* upravlja pripremom podataka za skladištenje i njihovom rekonstrukcijom. Njene funkcionalnosti uključuju:

- kreiranje TAR arhive,
- GZIP kompresiju podataka,
- podelu sadržaja na sektore,
- izračunavanje potrebnog prostora za smeštanje fajlova,
- raspakivanje i rekonstrukciju skrivenih fajlova.

6.4.3. StorageManager

Klasa *StorageManager* upravlja procesom skladištenja podataka u *slack* prostor i održavanjem SAT tabele. Njene odgovornosti su:

- alokacija slobodnih *slack* sektora,
- upis podataka na uređaj,
- čitanje podataka sa uređaja,
- brisanje sadržaja iz *slack* prostora,
- vođenje evidencije o iskorišćenim *slack* sektorima,
- čuvanje i učitavanje SAT tabele.

6.5. Implementacija

6.5.1. Pronalaženje slack prostora

Prvi korak u radu alata predstavlja određivanje putanje do USB uređaja na kome se nalazi fajl sistem u okviru koga se skrivaju podaci. Ova funkcionalnost realizovana je metodom *_get_device_path()*, koja se poziva u konstruktoru klase *DriveHandler*. Metoda koristi sistemsku komandu *df* kako bi na osnovu prosleđene *mount_path* putanje odredila odgovarajuću putanju uređaja (*dev_path*), na primer */dev/sdb1*. Dobijena informacija se kasnije koristi za pristup metapodacima fajl sistema i direktan rad sa fizičkim uređajem.

Nakon određivanja putanje do uređaja, vrši se pronalaženje svih fajlova koji se nalaze na ciljnoj particiji. Ova funkcionalnost implementirana je metodom *discover_all_files()*, koja koristi funkciju *os.walk()* za rekurzivni obilazak svih direktorijuma i poddirektorijuma. Tokom obilaska prikupljaju se putanje do svih pronađenih fajlova i smeštaju u internu listu *file_paths* koja će biti korišćena u daljoj analizi.

```
def discover_all_files(self):  
    print("[Drive handler info]: Discovering all files on the filesystem...")
```



```

file_count = 0

for directory, subdirs, filenames in os.walk(self.mount_path):
    for filename in filenames:
        full_path = os.path.join(directory, filename)
        self.file_paths.append(full_path)
        file_count += 1

print(f"[Drive handler info]: Found {file_count} files")

return self.file_paths

```

Kod 1. Pronalaženje svih fajlova na ciljnoj particiji USB uređaja

Nakon prikupljanja putanja do fajlova, pokreće se proces mapiranja *slack* prostora metodom *map_all_slack_space()*. Ova metoda prolazi kroz svaki pronađeni fajl i za njega poziva funkciju *analyze_file_for_slack_space()*, čiji je zadatak određivanje raspoloživog *slack* prostora.

Prvi korak analize jednog fajla metodom *analyze_file_for_slack_space()* podrazumeva pribavljanje osnovnih metapodataka korišćenjem funkcije *os.stat()*. Na taj način dobijaju se veličina fajla i broj *inode*-a koji predstavlja jedinstveni identifikator fajla unutar ext4 fajl sistema.

```

def analyze_file_for_slack_space(self, file_path):
    ...

    stat_info = os.stat(file_path)

    file_size = stat_info.st_size

    inode_number = stat_info.st_ino

    ...

```

Kod 2. Prikupljanje osnovnih metapodataka fajla

Dobijeni *inode* broj koristi se za pristup informacijama o fizičkoj organizaciji fajla na disku pomoću alata *debugfs*. Analizom *extent* zapisa, koji se dobija izvršavanjem *stat* komande, moguće je odrediti fizičku lokaciju poslednjeg bloka koji pripada određenom fajlu.

```

def analyze_file_for_slack_space(self, file_path):
    ...

```

```

cmd = ['debugfs', '-R', f'stat <{inode_number}>', self.device]

result = subprocess.run(cmd, capture_output=True, text=True)

extent_info = self._parse_extents(result.stdout)

...

```

Kod 3. Dobijanje *extent* informacije

Dobijeni izlaz zatim se obrađuje metodom `_parse_extents()`. Tokom obrade izdvajaju se fizičke adrese blokova koje pripadaju fajlu. Pošto se *slack* prostor nalazi na kraju poslednjeg alociranog bloka, za dalju analizu koristi se poslednji pronađeni *extent*. Nakon određivanja početnog i završnog bloka vrši se konverzija blokova u bajt adrese. Na osnovu lokacije poslednjeg bloka izračunava se stvarni kraj fajla i količina neiskorišćenog prostora unutar tog bloka. Ovaj prostor predstavlja *slack* space koji može biti iskorišćen za skladištenje skrivenih podataka.

```

def analyze_file_for_slack_space(self, file_path):
    ...

    # converting blocks to byte offsets
    extent_start_bytes = extent_start_block * self.block_size
    extent_end_bytes = (extent_end_block + 1) * self.block_size

    # actual ending of a file
    file_end_bytes = extent_start_bytes + file_size

    slack_start = file_end_bytes
    slack_end = extent_end_bytes
    slack_size = slack_end - slack_start

    ...

```

Kod 4. Izračunavanje granice *slack* prostora

Dobijeni opsezi *slack* prostora zatim se prevode u listu fizičkih sektora veličine jednog sektora koji je podrazumevano 512 bajta, metodom `_convert_ranges_to_sectors()`. Početna adresa svakog *slack* sektora dodaje u listu raspoloživih lokacija za skladištenje podataka *available_slack_locations*.

```

def _convert_ranges_to_sectors(self):

```

```
#converting slack space byte ranges to sector offsets

print("[Drive handler info]: Converting byte ranges to sector offsets...")

for slack_info in self.slack_space_info:
    slack_start = slack_info['slack_start']
    slack_end = slack_info['slack_end']

    for sector_offset in range(slack_start, slack_end, self.sector_size):
        if sector_offset < slack_end:
            self.available_slack_locations.append(sector_offset)

self.available_slack_locations.sort()

print(f"[Drive handler info]: Mapped {len(self.available_slack_locations)}
available sectors")
```

Kod 5. Konverzija *slack* prostora u listu raspoloživih sektora

6.5.2. Skrivanje podataka

Proces skrivanja podataka sastoji se iz nekoliko faza: pripreme podataka, određivanja potrebnog prostora za upis i fizičkog upisa u *slack* prostor.

Nakon pronalaženja raspoloživih sektora u *slack* prostoru, sledeći korak predstavlja priprema podataka za skladištenje. Ovaj deo funkcionalnosti implementiran je u okviru klase *FileHandler*, čiji je zadatak da ulazne fajlove pripremi za upis u prethodno mapirane sektore.

Korisnik kao ulaz može proslediti pojedinačne fajlove ili listu fajlova koje želi da sakrije. Svaki ulaz se najpre objedinjuje u TAR arhivu, nakon čega se vrši GZIP kompresija kako bi se smanjila količina prostora potrebna za skladištenje. Ovo je omogućeno metodom *create_tar_archive(files_to_hide)*.

U okviru metode *split_into_chunks(compressed_blob)* dobijeni kompresovani sadržaj koji predstavlja binarne podatke deli se na segmente veličine jednog *slack* sektora. Ukoliko poslednji segment ne popunjava ceo sektor, vrši se dopunjavanje nulama kako bi svi segmenti imali jednaku veličinu. Metoda vraća listu segmenata, kao i broj segmenata koje je potrebno upisati u *slack* prostor. Informacija o stvarnoj veličini kompresovanog sadržaja čuva se u SAT tabeli i koristi se tokom rekonstrukcije.

```
def split_into_chunks(self, compressed_blob):
```

```

        print(f"[File handler info]: Splitting blob into {self.sector_size}-byte
chunks...")

        chunks = []

        # Split blob into slack sector-sized pieces
        for i in range(0, len(compressed_blob), self.sector_size):
            chunk = compressed_blob[i:i + self.sector_size]

            # If the last chunk is smaller than sector_size, pad it with zeros
            if len(chunk) < self.sector_size:
                chunk = chunk + b'\x00' * (self.sector_size - len(chunk))

            chunks.append(chunk)

        print(f"[File handler info]: Created {len(chunks)} chunks")

        return chunks, len(chunks)

```

Kod 6. Podela podataka na segmente

Nakon pripreme podataka, poziva se metoda *write_to_slack()* klase *StorageManager*, koja određuje broj potrebnih sektora i pronalazi dovoljan broj slobodnih lokacija u prethodno formiranoj mapi *slack* prostora.

```

def write_to_slack(self, blob_id, blob_chunks, sectors_needed, original_size):

    free_sectors = [s for s in self.available_slack_sectors if s not in
self.used_sectors]
    print("Free sectors", free_sectors[0:7])

    if len(free_sectors) < sectors_needed:
        print(f"[Storage manager info]: Not enough slack space. Need
{sectors_needed}, have {len(free_sectors)}")
        return None

    allocated_sectors = free_sectors[:sectors_needed]
    print("Allocated sectors: ", allocated_sectors)

    first_sector = allocated_sectors[0]

```

```

        last_sector = allocated_sectors[-1]

        # write chunks to disk

        try:

            self._write_to_device(blob_chunks, allocated_sectors)

        except Exception as e:

            print(f"[Storage manager error]: Error writing to device: {e}")

        ...

```

Kod 7. Deo metode *write_to_slack* kojim se određuju slobodni *slack* sektori

Ukoliko postoji dovoljan broj slobodnih sektora, podaci se direktno upisuju na uređaj korišćenjem operacija *seek()* i *write()* u okviru metode *_write_to_device()*.

```

def _write_to_device(self, chunks, sectors):

    try:

        with open(self.device_path, 'r+b') as device:

            for chunk, sector_offset in zip(chunks, sectors):

                device.seek(sector_offset)

                device.write(chunk)

            print(f"[Storage manager info]: Successfully wrote {len(chunks)} sectors")

    except Exception as e:

        print(f"[Storage manager error]: Error writing to device: {e}")

```

Kod 8. Direktan upis podataka u *slack* sektore

Po završetku upisa kreira se zapis u SAT tabeli koji omogućava kasnije pronalaženje i rekonstrukciju podataka.

```

sat_entry = {

    'file_id': file_id,

    'first_sector': first_sector,

    'last_sector': last_sector,

    'num_sectors': sectors_needed,

```

```
'data_size': original_size,

'timestamp': datetime.now().isoformat()

}
```

Kod 9. Kreiranje zapisa u SAT tabeli

6.5.3. Evidencija skrivenih podataka korišćenjem SAT tabele

Pošto se skriveni podaci fizički nalaze van standardne strukture fajl sistema, potrebno je obezbediti mehanizam za njihovo kasnije pronalaženje. U tu svrhu implementirana je posebna tabela za evidenciju alokacije sektora.

SAT tabela predstavlja centralni mehanizam za upravljanje skrivenim podacima i sadrži informacije o svakom smeštenom fajlu:

- jedinstveni identifikator fajla,
- početni sektor,
- završni sektor,
- broj zauzetih sektora,
- originalnu veličinu podataka,
- vreme skladištenja.

Tabela se čuva u JSON formatu, što omogućava jednostavno učitavanje i ažuriranje prilikom pokretanja programa.

Prilikom pokretanja programa SAT tabela se učitava sa diska pozivom `_load_sat_from_disk()` u okviru konstruktora `StorageManager` klase i na osnovu nje se formira skup zauzetih sektora `used_sectors`. Ovim postupkom sprečava se mogućnost da novi podaci budu upisani preko već postojećih skrivenih sadržaja.

```
def _load_sat_from_disk(self):

    sat_table=[]

    try:

        with open(self.sat_file_path, 'r') as f:

            sat_table = json.load(f)

    self.used_sectors = set()

    for entry in sat_table:

        first = entry['first_sector']

        num = entry['num_sectors']

        for i in range(num):
```

```

        self.used_sectors.add(first + i*self.sector_size)

    print(f"[Storage manager info]: SAT loaded from {self.sat_file_path}
    ({len(sat_table)} files)")

except Exception as e:

    with open(self.sat_file_path, 'w') as f:

        json.dump(sat_table, f)

    print(f"[Storage manager error]: No available SAT, making a new one.")

return sat_table

```

Kod 10. Učitavanje SAT tabele iz JSON fajla i popunjavanje *used_sectors* seta

Nakon svake operacije skrivanja ili brisanja tabela se ažurira i ponovo zapisuje na disk metodom *save_sat_to_disk()*.

6.5.4. Rekonstrukcija podataka

Proces čitanja skrivenih podataka započinje pronalaženjem odgovarajućeg zapisa u SAT tabeli u okviru metode *read_from_slack(file_id)* na osnovu prosleđivanja identifikatora željenog fajla. Na osnovu podataka o lokaciji sektora određuju se fizičke adrese sa kojih treba izvršiti čitanje.

Sadržaj svih sektora koji pripadaju izabranom fajlu učitava se sa uređaja i spaja u jedinstveni binarni blok koji se vraća pozivom *_read_from_device(sectors_to_read, data_size)*. Pošto se prilikom skladištenja poslednji sektor dopunjava nulama, višak podataka se uklanja korišćenjem originalne veličine sadržaja sačuvane u SAT tabeli.

```

def read_from_slack(self, blob_id):

    # find the SAT entry

    sat_entry = None

    for entry in self.sat:

        if entry['file_id'] == blob_id:

            sat_entry = entry

            break

    if not sat_entry:

```

```

        print(f"[Storage manager info]: File '{blob_id}' not found in SAT")
        return None

    #read file from sectors
    first_sector = sat_entry['first_sector']
    last_sector = sat_entry['last_sector']
    data_size = sat_entry['data_size']

    sectors_to_read = [
        sector for sector in self.available_slack_sectors
        if first_sector <= sector <= last_sector
    ]

    file_data = self._read_from_device(sectors_to_read, data_size)

    print(f"[Storage manager info]: Read '{blob_id}' from sectors {first_sector}-
    {sat_entry['last_sector']}")
    return file_data

```

Kod 11. Pronalazak fajl u okviru SAT tabele i čitanje njegovog binarnog sadržaja

Pozivom metode *FileHandler* klase, *extract_files(blob, output_directory)*, vrši se GZIP dekompresija i raspakivanje TAR arhive, čime se dobijaju originalni fajlovi na putanji koju je korisnik definisao pomoću ulaznog parametra.

6.4.5. Brisanje podataka

Brisanje skrivenih podataka realizovano je prepisivanjem svih *slack* sektora koji pripadaju određenom fajlu nulama. Nakon uspešnog brisanja fizičkog sadržaja odgovarajući zapis se uklanja iz SAT tabele.

6.6. CLI interfejs

Alat je realizovan kao komandno-linijska aplikacija korišćenjem Python biblioteke *argparse*. Ovakav pristup omogućava jednostavno korišćenje alata bez potrebe za grafičkim korisničkim interfejsom.

Implementirane su četiri osnovne komande:

- `hide` – koristi se za skrivanje jednog ili liste fajlova u *slack* prostoru
- `read` – omogućava čitanje i rekonstrukciju prethodno skrivenih podataka
- `delete` – vrši uklanjanje podataka iz *slack* prostora i SAT tabele
- `list` – prikazuje informacije o trenutno skrivenim fajlovima.

U zavisnosti od odabrane komande pozivaju se odgovarajuće implementirane funkcije *read*, *hide*, *delete*, *list_hidden* koje instanciraju objekte *FileHandler*, *DriveHandler* i *StorageManager* klasa i pozivaju njihove metode kako bi se izvršili očekivani zadaci svake od funkcija.

6.7. Testiranje

Prvo je testirana funkcionalnost skrivanja podataka korišćenjem fajlova različitih veličina. Primer poziva kojima se sakrivaju 2 fajla *file1.txt* i *file2.pdf* može se videti u nastavku.

```
sudo python3 slack_tool.py hide -d /dev/sdb1 -f file1.txt file2.pdf -m /mnt/testusb
-s ./sat.json
```

Kod 12. Primer poziva alata radi skrivanja fajlova

Rezultati su pokazali da se podaci uspešno kompresuju, dele na sektore i smeštaju u raspoložive lokacije *slack* prostora što se može videti u logovima koje vraća alat na slici 5.

```
Hiding ['file1.txt', 'file2.pdf'] on /dev/sdb1
[Drive handler info]: Device identified - /dev/sdb1
[Drive handler info]: Discovering all files on the filesystem...
[Drive handler info]: Found 205 files
[Drive handler info]: Mapping slack space across all files...
[Drive handler info]: Found 205 slack cluster spaces
[Drive handler info]: Total slack space available: 0.40 MB
[Drive handler info]: Converting byte ranges to sector offsets...
[Drive handler info]: Mapped 906 available sectors
[Storage manager info]: SAT loaded from ./sat.json (0 files)
[File handler info]: Creating tar.gz archive from 1 file(s)...
[File handler info] Uncompressed tar size: 10.00 KB
[File handler info]: Compressed tar.gz size: 1.37 KB
[File handler info]: Compression ratio: 86.3%
[File handler info]: Splitting blob into 512-byte chunks...
[File handler info]: Created 3 chunks
Free sectors [136245582, 136246094, 136246606, 136247118, 136247630, 136248142, 136248654]
Allocated sectors: [136245582, 136246094, 136246606]
[Storage manager info]: Successfully wrote 3 sectors
[Storage manager info]: Wrote 'file1.txt' to sectors 136245582-136246606 (3 sectors)
[File handler info]: Creating tar.gz archive from 1 file(s)...
[File handler info] Uncompressed tar size: 10.00 KB
[File handler info]: Compressed tar.gz size: 0.20 KB
[File handler info]: Compression ratio: 98.0%
[File handler info]: Splitting blob into 512-byte chunks...
[File handler info]: Created 1 chunks
Free sectors [136247118, 136247630, 136248142, 136248654, 136249166, 136249678, 136250190]
Allocated sectors: [136247118]
[Storage manager info]: Successfully wrote 1 sectors
[Storage manager info]: Wrote 'file2.pdf' to sectors 136247118-136247118 (1 sectors)
[Storage manager info]: SAT saved to ./sat.json
```

Slika 5. Izlaz alata tokom procesa skrivanja podataka

Na slici 6. se može videti sadržaj SAT tabele nakon skrivanja podataka.

```
{ } sat.json > ...
1  [
2      {
3          "file_id": "file1.txt",
4          "first_sector": 136245582,
5          "last_sector": 136246606,
6          "num_sectors": 3,
7          "data_size": 1399,
8          "timestamp": "2026-07-14T19:00:31.018108"
9      },
10     {
11         "file_id": "file2.pdf",
12         "first_sector": 136247118,
13         "last_sector": 136247118,
14         "num_sectors": 1,
15         "data_size": 200,
16         "timestamp": "2026-07-14T19:00:31.020805"
17     }
18 ]
```

Slika 6. Sadržaj SAT tabele nakon skrivanja podataka

Izvršen je poziv u nastavku kako bi bila proverena komanda *list*, a rezultati izvršenja su prikazani na slici 7.

```
sudo python3 slack_tool.py list -d /dev/sdb1 -s ./sat.json -m /mnt/testusb
```

Kod 13. Primer poziva alata radi listanja svih skrivenih fajlova

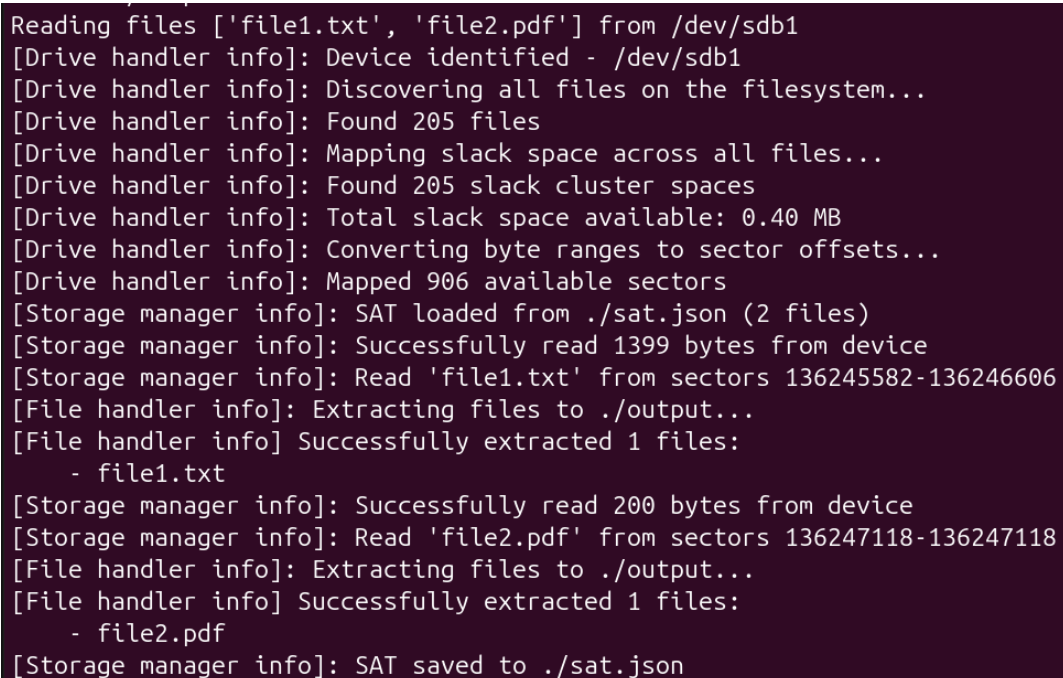
```
Listing hidden files on /dev/sdb1
[Drive handler info]: Device identified - /dev/sdb1
[Drive handler info]: Discovering all files on the filesystem...
[Drive handler info]: Found 205 files
[Drive handler info]: Mapping slack space across all files...
[Drive handler info]: Found 205 slack cluster spaces
[Drive handler info]: Total slack space available: 0.40 MB
[Drive handler info]: Converting byte ranges to sector offsets...
[Drive handler info]: Mapped 906 available sectors
[Storage manager info]: SAT loaded from ./sat.json (2 files)
[Storage manager info]: Hidden files in slack space:
-----
ID: file1.txt
  Location: sectors 136245582-136246606
  Size: 1399 bytes (3 sectors)
  Added: 2026-07-14T19:00:31.018108
ID: file2.pdf
  Location: sectors 136247118-136247118
  Size: 200 bytes (1 sectors)
  Added: 2026-07-14T19:00:31.020805
-----
[Storage manager info]: SAT saved to ./sat.json
```

Slika 7. Rezultati izvršenja *list* komande

Funkcionalnost čitanja testirana je pozivom u nastavku, a poređenjem originalnih i rekonstruisanih fajlova u okviru *output* datoteke potvrđeno je da tokom procesa nije došlo do izmene ili gubitka podataka. Na slici 8 se može videti rezultat izvršenja.

```
sudo python3 slack_tool.py read -d /dev/sdb1 -f file1.txt file2.pdf -m /mnt/testusb  
-s ./sat.json -o ./output
```

Kod 14. Primer poziva alata radi čitanja skrivenih fajlova



```
Reading files ['file1.txt', 'file2.pdf'] from /dev/sdb1  
[Drive handler info]: Device identified - /dev/sdb1  
[Drive handler info]: Discovering all files on the filesystem...  
[Drive handler info]: Found 205 files  
[Drive handler info]: Mapping slack space across all files...  
[Drive handler info]: Found 205 slack cluster spaces  
[Drive handler info]: Total slack space available: 0.40 MB  
[Drive handler info]: Converting byte ranges to sector offsets...  
[Drive handler info]: Mapped 906 available sectors  
[Storage manager info]: SAT loaded from ./sat.json (2 files)  
[Storage manager info]: Successfully read 1399 bytes from device  
[Storage manager info]: Read 'file1.txt' from sectors 136245582-136246606  
[File handler info]: Extracting files to ./output...  
[File handler info]: Successfully extracted 1 files:  
- file1.txt  
[Storage manager info]: Successfully read 200 bytes from device  
[Storage manager info]: Read 'file2.pdf' from sectors 136247118-136247118  
[File handler info]: Extracting files to ./output...  
[File handler info]: Successfully extracted 1 files:  
- file2.pdf  
[Storage manager info]: SAT saved to ./sat.json
```

Slika 8. Rezultat izvršenja čitanja skrivenih fajlova

Posebno je testirana funkcija brisanja jednog od skrivenih fajlova. Nakon izvršavanja operacije potvrđeno je da je zapis uklanjen iz SAT tabele, čime prethodno skriveni podaci više nisu dostupni za jednostavnu rekonstrukciju. Na slici 9 se može videti novi sadržaj SAT tabele, dok se na slici 10 može videti rezultat izvršenja operacije.

```
sudo python3 slack_tool.py delete -d /dev/sdb1 -f file1.txt -m /mnt/testusb -s  
./sat.json
```

Kod 15. Primer poziva alata radi brisanja skrivenog fajla

```

{} sat.json > ...
1  [
2    {
3      "file_id": "file2.pdf",
4      "first_sector": 136247118,
5      "last_sector": 136247118,
6      "num_sectors": 1,
7      "data_size": 200,
8      "timestamp": "2026-07-14T19:00:31.020805"
9    }
10 ]

```

Slika 9. Sadržaj SAT tabele

```

Deleting files ['file1.txt'] from /dev/sdb1
[Drive handler info]: Device identified - /dev/sdb1
[Drive handler info]: Discovering all files on the filesystem...
[Drive handler info]: Found 205 files
[Drive handler info]: Mapping slack space across all files...
[Drive handler info]: Found 205 slack cluster spaces
[Drive handler info]: Total slack space available: 0.40 MB
[Drive handler info]: Converting byte ranges to sector offsets...
[Drive handler info]: Mapped 906 available sectors
[Storage manager info]: SAT loaded from ./sat.json (2 files)
sectors to clear [136245582, 136246094, 136246606]
[Storage manager info]: Deleted 'file1.txt' from sectors 136245582-136246606
[Storage manager info]: SAT saved to ./sat.json

```

Slika 10. Rezultat izvršenja operacije brisanja skrivenog fajla

6.8. Ograničenja

Razvijeni alat poseduje određena ograničenja koja proizlaze iz načina implementacije i karakteristika korišćenog fajl sistema.

Prvo ograničenje odnosi se na podršku za fajl sisteme. Trenutna implementacija razvijena je samo za ext4 fajl sistem i oslanja se na alat *debugfs*, zbog čega nije direktno primenljiva na NTFS, FAT32, exFAT ili druge fajl sisteme bez dodatnih izmena.

Drugo ograničenje predstavlja potreba za administratorskim privilegijama. Direktan pristup uređaju i analiza metapodataka fajl sistema zahtevaju odgovarajuće privilegije operativnog sistema.

Treće ograničenje odnosi se na pouzdanost skladištenja podataka. Ukoliko dođe do izmene ili proširenja fajlova čiji je *slack* prostor iskorišćen za skladištenje podataka, postoji mogućnost prepisivanja skrivenog sadržaja i njegovog trajnog gubitka.

7. Zaključak

U okviru ovog rada obrađena je tema forenzike fajl sistema, sa posebnim osvrtom na tehnike skrivanja podataka i implementaciju alata za skrivanje podataka korišćenjem *slack space*-a.

U uvodnom delu rada predstavljen je značaj digitalne forenzike kao discipline koja omogućava prikupljanje, analizu i interpretaciju digitalnih dokaza na način prihvatljiv u pravnom kontekstu. Kroz drugo poglavlje objašnjeni su osnovni principi digitalne forenzike, njen praktični i teorijski značaj, kao i strukturiran proces forenzičke analize digitalnih dokaza definisan od strane NIST-a, koji obuhvata prikupljanje, ispitivanje, analizu i izveštavanje o digitalnim podacima. Treće poglavlje posvećeno je osnovama fajl sistema, gde su opisane njihove osnovne funkcije, komponente, kao i način organizacije i alokacije podataka na disku. Posebna pažnja posvećena je pojmovima nealociranog prostora i *slack space*-a, koji predstavljaju ključne oblasti od interesa za digitalnu forenziku, budući da mogu sadržati ostatke prethodno obrisanih ili namerno skrivenih podataka. U četvrtom poglavlju analizirani su najpoznatiji fajl sistemi, FAT, exFAT, NTFS, HFS, APFS i Ext4, kroz njihove osnovne karakteristike, prednosti, nedostatke i forenzički potencijal, čime je pokazano da izbor fajl sistema značajno utiče na mogućnosti forenzičke analize i oporavka podataka. Peto poglavlje obuhvatilo je pregled najčešćih tehnika za skrivanje podataka u okviru fajl sistema, uključujući hidden atribut fajlova, skrivanje u nealociranom prostoru, Alternate Data Streams, steganografiju na nivou fajl sistema, skrivanje u metapodacima i skrivanje u *slack space*-u. Poređenjem ovih tehnika utvrđeno je da svaka od njih ima specifične prednosti i ograničenja u pogledu kapaciteta, kompleksnosti implementacije i otpornosti na forenzičku detekciju, te da kombinovanje više tehnika, uz primenu kriptografskih metoda, dodatno otežava otkrivanje skrivenih podataka.

Na osnovu teorijskih osnova iznetih u prethodnim poglavljima, u šestom poglavlju razvijen je i opisan CLI alat za skrivanje podataka u *slack* prostoru ext4 fajl sistema. Alat omogućava skrivanje, čitanje, brisanje i pregled skrivenih fajlova, uz vođenje evidencije o iskorišćenom prostoru putem SAT tabele. Testiranjem je potvrđeno da alat uspešno skriva i rekonstruiše podatke bez gubitka ili izmene sadržaja, čime je praktično demonstriran koncept *slack space*-a kao pogodne lokacije za skrivanje podataka, ali i istaknuta ograničenja ovakvog pristupa, pre svega u pogledu kapaciteta i rizika od prepisivanja skrivenog sadržaja.

Sprovedeno istraživanje i razvijeno rešenje potvrđuju da *slack space*, iako naizgled zanemarljiv deo fajl sistema, predstavlja značajan izvor digitalnih tragova i potencijalnu pretnju u pogledu neovlašćenog skrivanja podataka. Dalji pravci razvoja mogli bi obuhvatiti proširenje alata na dodatne fajl sisteme, kao i implementaciju naprednijih mehanizama zaštite, poput enkripcije i dinamičkog rasporeda skrivenih podataka.

8. Reference

- [1] **IBM Think – Digital Forensics**, <https://www.ibm.com/think/topics/digital-forensics>, *poslednji put posećeno: 18.07.2026.*
- [2] **GeeksforGeeks – File Allocation Methods in Operating System**, <https://www.geeksforgeeks.org/operating-systems/file-allocation-methods/>, *poslednji put posećeno: 18.07.2026.*
- [3] **Kingston Technology – Understanding File Systems**, <https://www.kingston.com/en/blog/personal-storage/understanding-file-systems>
Poslednji put posećeno: 18.07.2026.
- [4] **GeeksforGeeks – FAT Full Form**, <https://www.geeksforgeeks.org/operating-systems/fat-full-form/>, *poslednji put posećeno: 18.07.2026.*
- [5] **7 Data Recovery Glossary – HFS**, <https://glossary.7datarecovery.com/hfs/>, *poslednji put posećeno: 18.07.2026.*
- [6] **MSAB Glossary – APFS (Apple File System)**, <https://www.msab.com/glossary/apfs-apple-file-system/>, *poslednji put posećeno: 18.07.2026.*
- [7] **MSAB Glossary – EXT4 (Fourth Extended File System)**, <https://www.msab.com/glossary/ext4-fourth-extended-file-system/>, *poslednji put posećeno: 18.07.2026.*